Efficient Maintenance of Leiden Communities in Large Dynamic Graphs

Anonymous Author(s)

Abstract

As a well-known community detection algorithm, Leiden has been widely used in various scenarios such as large language model (LLM) generation, anomaly detection, and biological analysis. In these scenarios, the graphs are often large and dynamic, where vertices and edges are inserted and deleted frequently, so it is costly to obtain the updated communities by Leiden from scratch when the graph has changed. Recently, one work has attempted to study how to maintain Leiden communities in the dynamic graph, but it lacks a detailed theoretical analysis, and its algorithms are inefficient for large graphs. To address these issues, in this paper, we first theoretically show that the existing algorithms are relatively unbounded via the boundedness analysis (a powerful tool for analyzing incremental algorithms on dynamic graphs), and also analyze the memberships of vertices in communities when the graph changes. Based on theoretical analysis, we develop a novel efficient maintenance algorithm, called *Hierarchical Incremental Tree Leiden* (`HIT-Leiden`), which effectively reduces the range of affected vertices by maintaining the connected components and hierarchical community structures. Comprehensive experiments in various datasets demonstrate the superior performance of `HIT-Leiden`. In particular, it achieves speedups of up to five orders of magnitude over existing methods.

CCS Concepts

• **Information systems** → **Clustering**; **Data stream mining**; • **Theory of computation** → **Dynamic graph algorithms**.

Keywords

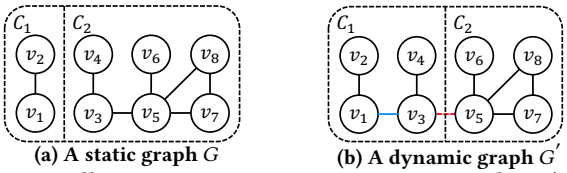
Incremental graph algorithms, community detection, Leiden algorithm

ACM Reference Format:

Anonymous Author(s). 2026. Efficient Maintenance of Leiden Communities in Large Dynamic Graphs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (SIGMOD '26).* ACM, New York, NY, USA, 17 pages. https://doi.org/XXXXXXX.XXXXXXX

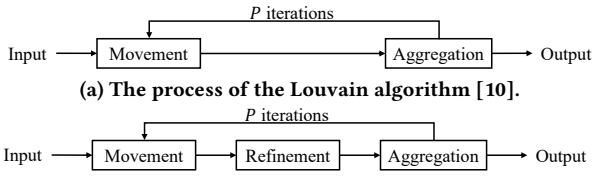
1 Introduction

As one of the fundamental measures in network science, modularity [60] effectively measures the strength of division of a network into modules (also called communities). Essentially, it captures the difference between the actual number of edges within a community and the expected number of such edges if connections were random. By maximizing the modularity of a graph, it can reveal all the communities in the graph. In Figure 1(a), for example, by maximizing the modularity of the graph, we can obtain two communities C_1 and C_2. As shown in the literature [13, 78], the graph communities have found a wide range of applications in recommendation systems, social marketing, and biological analysis.



(a) A static graph G **(b) A dynamic graph G'**

Figure 1: Illustrating community maintenance, where (v_1, v_3) is a newly inserted edge and (v_3, v_5) is a newly deleted edge.



(a) The process of the Louvain algorithm [10].

(b) The process of the Leiden algorithm [80].

Figure 2: Illustrating the Louvain and Leiden algorithms.

One of the most popular community detection (CD) algorithms that use modularity maximization is Louvain [10], which partitions a graph into disjoint communities. As shown in Figure 2(a), Louvain employs an iterative process with each iteration having two phases, called ***movement*** and ***aggregation***, to adjust the community structure and improve modularity. Specifically, in the movement phase, each vertex is relocated to a suitable community to maximize the modularity of the graph. In the aggregation phase, all the vertices belonging to the same community are merged into a supervertex to form a supergraph for the next iteration. Since a supervertex corresponds to a set of vertices, the communities of a graph naturally form a tree-like hierarchical structure. In practice, to balance modularity gains against the running time, users often limit Louvain to P iterations, where P is a pre-defined parameter.

Despite its popularity, Louvain may produce communities that are internally disconnected. This typically occurs during the movement phase, where a vertex that serves as a bridge within a community may be moved to a different community that has stronger connections, thereby breaking the connectivity of the original community. To overcome this issue, Traag et al. [80] proposed the *Leiden* algorithm [1], which introduces an additional phase, called ***refinement***, between the movement and aggregation phases, as shown in Figure 2(b). Specifically, during the refinement phase, vertices explore merging with their neighbors within the same community to form sub-communities. By adding this additional phase, Leiden produces communities with higher quality than Louvain, since its communities well preserve the connectivity.

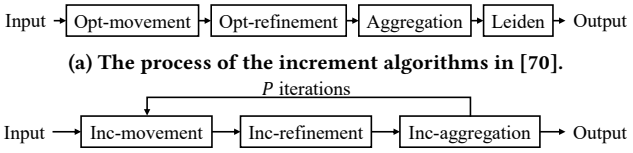
[1] As of July 2025, Leiden has received over 5,000 citations according to Google Scholar.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX



(a) The process of the increment algorithms in [70].

(b) The process of our `HIT-Leiden` algorithm.

Figure 3: Algorithms for maintaining Leiden communities.

As shown in the literature, Leiden has recently received plenty of attention because of its applications in many areas, including large language model (LLM) generation [43, 54, 55, 63, 104], anomaly detection [27, 38, 65, 73, 82], and biological analysis [1, 8, 28, 47, 99]. For example, Microsoft has recently developed Graph-RAG [54], a retrieval-augmented generation (RAG) method that enhances prompts by searching external knowledge to improve the accuracy and trustworthiness of LLM generation, and builds a hierarchical index by using the communities detected by Leiden. As another example, Liu et al. introduced eRiskComm [48], a community-based fraud detection system that assists regulators in identifying high-risk individuals from social networks by using Louvain to partition communities, and Leiden can be naturally applied in this context.

In the aforementioned application scenarios, the graphs often evolve frequently over time, with many insertions and deletions of vertices and edges. For instance, in Wikipedia, the number of English articles increases by about 15,000 per month as of July 2024[2], making their contributors form a massive and continuously evolving collaboration graph, where nodes represent users. In these settings, changes to the underlying graph can significantly alter the communities produced by Leiden, thereby affecting downstream tasks and decision-making. However, the original Leiden algorithm is designed for static graphs, so it is very costly to recompute the communities from scratch using Leiden whenever a graph change occurs, especially for large graphs. Hence, it is strongly desirable to develop efficient algorithms for maintaining the up-to-date Leiden communities in large dynamic graphs.

Prior works. To maintain Louvain communities in dynamic graphs, several algorithms have been developed, such as DF-Louvain [69], Delta-Screening [97], DynaMo [105], and Batch [18]. However, little attention has been paid to maintaining Leiden communities. To the best of our knowledge, [70] is the only work that achieves this. It first uses some optimizations for the first iteration of `DF-Leiden`, and then invokes the original Leiden algorithm for the remaining iterations, as depicted in Figure 3(a). Following the optimized movement phase (`opt-movement`), the refinement phase in DF-Leiden separates communities affected by edge or vertex changes into multiple sub-communities, while leaving unchanged communities as single sub-communities. The aggregation phase remains identical to that of the Leiden algorithm. After constructing the aggregated graph, the standard Leiden algorithm is applied to complete the remaining CD process. The author has also developed two variants of `DF-Leiden`, called `ND-Leiden` and `DS-Leiden`, by using different optimizations for the movement phase of the first iteration. Nevertheless, there is a lack of detailed theoretical analysis for these algorithms, and they are inefficient for large graphs with few changes.

[2]https://en.wikipedia.org/wiki/Wikipedia:Size_of_Wikipedia

Our contributions. To address the above limitations, we first theoretically analyze the time cost of existing algorithms for maintaining Leiden communities and theoretically show that they are relatively unbounded via the boundedness analysis, which is a powerful tool for analyzing the time complexity of incremental algorithms on dynamic graphs. We further analyze the membership of vertices in communities and sub-communities when the graph edges change, and observe that the procedure for maintaining these memberships generalizes naturally to all the supergraphs generated by Leiden. The above analysis not only lays a solid foundation for us to comprehend existing algorithms but also offers us opportunities to improve upon them.

Based on the above analyses, we develop a novel efficient maintenance algorithm, called Hierarchical Incremental Tree Leiden (HIT-Leiden), which effectively reduces the range of affected vertices by maintaining the connected components and hierarchical community structures. As depicted in Figure 3(b), `HIT-Leiden` is an iterative algorithm with each iteration having three key phases, namely incremental movement, incremental refinement, and incremental aggregation, abbreviated as `inc-movement`, `inc-refinement`, and `inc-aggregation`, respectively. More specifically, `inc-movement` extends the movement phase from [70] by incorporating hierarchical community structures [80]. Unlike prior approaches, it operates on a supergraph where each supervertex represents a sub-community, focusing on hierarchical dependencies between communities and their nested substructures. Inspired by the key technique of maintaining the connected components in dynamic graphs [90], `inc-refinement` maintains sub-communities by using tree-based structures to efficiently track changes in sub-communities. `Inc-aggregation` updates the supergraph by computing structural changes based on the outputs of the previous two phases.

We have evaluated `HIT-Leiden` on several large-scale real-world dynamic graph datasets. The experimental results show that our algorithm achieves comparable community quality with the state-of-the-art algorithms for maintaining Leiden communities, while achieving up to five orders of magnitude faster than `DF-Leiden`. In addition, we have deployed our algorithm in real-world applications at Company A[3].

Outline. We first review related work in Section 2. We then formally introduce some preliminaries, including the Leiden algorithm and problem definition in Section 3, provide some theoretical analysis in Section 4, and present our proposed `HIT-Leiden` algorithm in Section 5. Finally, we present the experimental results in Section 6 and conclude in Section 7.

2 Related Work

In this section, we first review the existing works of CD for both static and dynamic graphs. We simply classify these works as modularity and other metrics-based CD methods.

• **Modularity-based CD.** Modularity-based CD methods aim to partition a graph such that communities exhibit high internal connectivity relative to a null model. Among these methods, Louvain [10] is the most popular one due to its high efficiency and scalability as shown in some comparative analyses [4, 39, 94]. Leiden [80] improves upon Louvain by resolving the problem of disconnected communities, yielding higher-quality results with comparable runtime.

[3]Due to the SIGMOD double-anonymity policy, we cannot explicitly write the company name here, but we will show the name after acceptance.

Other modularity heuristics [19, 56, 58] or incorporate simulated annealing [11, 37], spectral techniques [59], and evolutionary strategies [42, 49]. Further refinements explore multi-resolution [77], robust optimization [5], normalized modularity [52], and clustering cost frameworks [35]. Recent neural approaches have integrated modularity objectives into deep learning models [9, 12, 89, 93, 100], enhancing representation learning for CD.

Besides, some recent works have studied how to incrementally maintain modularity-based communities when the graph is changed. Aynaud et al. [6] proposed one of the earliest approaches by reusing previous community assignments to warm-start the Louvain algorithm. Subsequent works extended this idea to both Louvain [18, 20, 53, 62, 69, 74, 75, 97] and Leiden [70], incorporating mechanisms such as edge-based impact screening or localized modularity updates. Nevertheless, the existing algorithms of maintaining Leiden communities lack in-depth theoretical analysis, and their practical efficiency is poor. Other methods based on modularity, including extensions to spectral clustering [17], multi-step CD [7], and label propagation-based methods [61, 86–88] have been studied on dynamic graphs.

• **Other metrics-based CD.** Beyond modularity, various CD methods have been developed by using different optimization purposes, such as similarity, statistical inference, spectral clustering, and neural networks. The similarity-based methods like SCAN [23, 83, 92] identify dense regions from the graph via structural similarity. Statistical inference approaches, including stochastic block models [2, 29, 36, 64], infer communities by fitting generative probabilistic models to observed networks. Spectral clustering methods [3, 22, 57] exploit the eigenstructure of graph Laplacians to group nodes with similar structural roles. Deep learning-based methods for CD have recently gained traction. Graph convolutional networks [21, 31, 32, 40, 50, 76, 91, 101, 103], and graph attention networks [26, 34, 51, 81, 84, 96] have demonstrated strong performance in learning expressive node embeddings for CD tasks. For more details, please refer to recent survey papers of CD [13, 78].

Besides, many of the above methods have also been extended for dynamic graphs. Ruan et al. [68] and Zhang et al. [98] have studied structural graph clustering on dynamic graphs, which is based on structural similarity. Temporal spectral methods [16, 17] and dynamic stochastic block models [45, 72] enable statistical modeling of evolving community structures over time. Recent deep learning approaches also support dynamic CD through mechanisms such as temporal embeddings [102], variational inference [41], contrastive learning [15, 24, 85], and generative modeling [33]. These models capture temporal dependencies and structural evolution.

3 Preliminaries

In this section, we first formally present the problem we study, and then briefly introduce the original Leiden algorithm. Table 1 summarizes the notations frequently used throughout this paper.

3.1 Problem definition

We consider an **undirected and weighted graph** $G = (V, E)$, where V and E are the sets of vertices and edges, respectively. Each vertex v's neighbor set is denoted by $N(v)$. Each edge (v_i, v_j) is associated with a positive weight $w(v_i, v_j) > 0$. The degree of v_i is given by $d(v_i) = \sum_{v_j \in N(v_i)} w(v_i, v_j)$. Denote by m the total weight of all edges in G, i.e., $m = \sum_{(v_i,v_j)\in E} w(v_i, v_j)$.

Table 1: Frequently used notations and their meanings.

Notation	Meaning
$G = (V, E)$	A graph with vertex set V and edge set E
$N(v)$, $N_2(v)$	The vertex v's 1- and 2-hop neighbor sets, resp.
$w(v_i, v_j)$	The weight of edge between v_i and v_j
$d(v)$	The weighted degree of vertex v
m	The total weight of all edges in G
$\mathbb{C}$	A set of communities forming a partition of G
Q	The modularity of the graph G with partition $\mathbb{C}$
$G^p = (V^p, E^p)$	The supergraph in the p-th iteration of Leiden
$\Delta Q(v \to C', \gamma)$	Modularity gain by moving v from C to C' with γ
$f(\cdot): V \to \mathbb{C}$	A mapping from vertices to communities
$f^p(\cdot): V^P \to \mathbb{C}$	A mapping from supervertices to communities
$s^p(\cdot)$: $V^p \to V^{p+1}$	A mapping from supervertices in p-th level to supervertices in $(p+1)$-th level (sub-communities)
ΔG	The set of changed edges in the dynamic graph

Given a graph $G = (V, E)$, the CD process aims to partition all the vertices of V into some disjoint sets $\mathbb{C}$, each of which is called a community, corresponding to a set of vertices that are densely connected. This process can be modeled as a mapping function $f(\cdot) : V \to \mathbb{C}$, such that each v belongs to a community $f(v)$ of the partition $\mathbb{C}$. For each vertex v, the total weight between v and a community C is denoted by $w(v, C) = \sum_{v' \in N(v)\cap C} w(v, v')$.

As a well-known CD metric, the modularity measures the difference between the actual number of edges in a community and the expected number of such edges.

Definition 1 (Modularity [10]). *Given a graph $G = (V, E)$ and a community partition $\mathbb{C}$ over V, the modularity $Q(G, \mathbb{C}, \gamma)$ of the graph G with the partition $\mathbb{C}$ is defined as:*

$$Q(G, \mathbb{C}, \gamma) = \sum_{C\in\mathbb{C}} \left(\frac{1}{2m} \sum_{v\in C} w(v, C) - \gamma \left(\frac{d(C)}{2m} \right)^2 \right), \tag{1}$$

where $d(C)$ is the total degree of all vertices in a community C, and $\gamma > 0$ is a superparameter.

Note that the parameter $\gamma > 0$ controls the granularity of the detected communities [67]. A higher γ favors smaller, finer-grained communities. In practice, γ is often set to 0.5, 1, 4, or 32, as shown in [46]. Besides, to guide community updates, the concept of modularity gain is often used to capture the changed modularity when a vertex is moved from one community to another.

Definition 2 (Modularity gain [10]). *Given a graph G, a partition $\mathbb{C}$, and a vertex v that belongs to a community C, the modularity gain of moving v from C to another community C' is defined as:*

$$\Delta Q(v \to C', \gamma) = \frac{w(v, C') - w(v, C)}{2m} + \frac{\gamma \cdot d(v) \cdot (d(C) - d(v) - d(C'))}{(2m)^2}. \tag{2}$$

In this paper, we focus on the dynamic graph with insertions and deletions of both vertices and edges. Since a vertex insertion (resp. deletion) can be modeled as a sequence of edge insertions (resp. deletions), we simply focus on edge changes. Given a set of edge changes ΔG to a graph $G = (V, E)$, we obtain an updated graph $G' = (V', E')$. Since there are two types of edge updates, we let $\Delta G = \Delta G_+ \cup \Delta G_-$, where $\Delta G_+ = E' \setminus E$ and $\Delta G_- = E \setminus E'$ denote the sets of inserted and deleted edges, respectively. We denote updated edges $(v_i, v_j, \alpha) \in \Delta G_+$ and $(v_i, v_j, -\alpha) \in \Delta G_-$, where α is positive,

Algorithm 1: Leiden algorithm [71, 79]

Input: $G, f(\cdot), P, \gamma$
Output: Updated $f(\cdot)$
1 $G^1 \leftarrow G, f^1(\cdot) \leftarrow f(\cdot)$;
2 **for** $p = 1$ *to* P **do**
3 $\quad f^p(\cdot) \leftarrow Move(G^p, f^p(\cdot), \gamma)$;
4 $\quad s^p(\cdot) \leftarrow Refine(G^p, f^p(\cdot), \gamma)$;
5 $\quad$ **if** $p < P$ **then**
6 $\quad\quad G^{p+1}, f^{p+1}(\cdot) \leftarrow Aggregate(G^p, f^p(\cdot), s^p(\cdot))$;
7 Update $f(\cdot)$ using $s^1(\cdot), \cdots, s^P(\cdot)$;
8 **return** $f(\cdot)$;

i.e., $\alpha > 0$. We use $G \oplus \Delta G$ to denote applying ΔG to G, yielding an updated graph G'.

We now formally introduce the problem studied in this paper.

Problem 1 (Maintenance of Leiden communities [70]). *Given a graph G with its Leiden communities $\mathbb{C}$, and some edge updates ΔG, return the updated Leiden communities after applying ΔG to G.*

We illustrate our problem via Example 1.

Example 1. *In Figure 1(a), the original graph G with unit edge weights contains two Leiden communities: $C_1 = \{v_1, v_2\}$ and $C_2 = \{v_3, v_4, v_5, v_6, v_7, v_8\}$. After inserting a new edge (v_1, v_3) and deleting an existing edge (v_3, v_5) into G, we obtain an updated graph G', which has two updated communities $C_1 = \{v_1, v_2, v_3, v_4\}$ and $C_2 = \{v_5, v_6, v_7, v_8\}$.*

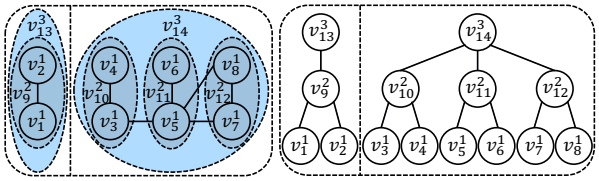
3.2 Leiden algorithm

Algorithm 1 presents Leiden [71, 79], following the process in Figure 2(b). Given a graph G, and an initial mapping $f(\cdot)$ (w.l.o.g., $f(v) = \{v\}$), it first initializes the level-1 supergraph G^1, lets level-1 mapping $f^1(\cdot)$ be $f(\cdot)$, and sets up the sub-community mapping $s(\cdot)$ (line 1). Next, it iterates P times, each having three phases.

(1) **Movement phase** (line 3): for each supervertex v^p in the supergraph G^p, it attempts to move v^p to a neighboring community that yields the maximum positive modularity gain, resulting in an updated community mapping $f^p(\cdot)$.
(2) **Refinement phase** (line 4): it splits each community into some sub-communities such that each of them corresponds to a connected component, producing a sub-community mapping $s^p(\cdot)$.
(3) **Aggregation phase** (line 6): when $p < P$, it aggregates each sub-community as a supervertex and builds a new graph G^{p+1}.

Finally, after P iterations, we update $f(\cdot)$ and obtain the communities (lines 7-8). Note that $f(\cdot)$ is updated using $s^P(\cdot)$ rather than $f^P(\cdot)$ since sub-communities guarantee connectivity with comparable modularity. Besides, we use the terms supervertex and sub-community interchangeably in this paper. A superedge is an edge between two supervertices, and its weight is the sum of the weights of edges between the supervertices.

Clearly, the vertices assigned to a sub-community will be further aggregated as a supervertex, so all the vertices and supervertices generated naturally form a tree-like hierarchical structure. The total time complexity of Leiden is $O(P \cdot (|V| + |E|))$ [71], since each iteration costs $O(|V| + |E|)$ time.



(a) All the communities. (b) A tree-like structure.

Figure 4: The process of Leiden for the graph G in Figure 1(a).

Example 2. *Figure 4 (a) depicts the process of Leiden with P=3 for the graph in Figure 1. Denote by v_i^p the supervertex (i.e., sub-community) in the p-th iteration of Leiden. It generates three levels of supergraphs: G^1, G^2, and G^3, with $G^1 = G$. The vertices of these supergraphs form a tree-like structure, as shown in Figure 4(b).*

Take the first iteration as an example depicted in Figure 5. In the movement phase, it generates three communities $C_1 = \{v_1^1, v_2^1\}$, $C_2 = \{v_5^1, v_6^1, v_7^1, v_8^1\}$ and $C_3 = \{v_3^1, v_4^1\}$. In the refinement phase, C_2 is split into two sub-communities $v_{11}^2 = \{v_5^1, v_6^1\}$ and $v_{12}^2 = \{v_7^1, v_8^1\}$, and C_1 and C_2 are unchanged. In the aggregation phase, all vertices are aggregated into supervertices based on their sub-community memberships, resulting in G^2.

4 Theoretical Analysis of Leiden

In this section, we first analyze the boundedness of existing algorithms, then study how vertex behavior impacts community structure under graph updates, and extend it to supergraphs.

4.1 Boundedness analysis

We first introduce some concepts related to boundedness.

• **Notation.** Let Θ denote the CD query applied to a graph G, where $\Theta(G) = \mathbb{C}$ is the set of detected communities. The new graph is $G \oplus \Delta G$, and the updated community is $\Theta(G \oplus \Delta G)$. We denote the output difference as $\Delta\mathbb{C}$, where $\Theta(G \oplus \Delta G) = \Theta(G) \oplus \Delta\mathbb{C}$.

• **Concepts of boundedness.** The notion of boundedness [66] evaluates the effectiveness of an incremental algorithm using the metric CHANGED, defined as CHANGED $= \Delta G + \Delta\mathbb{C}$, which leads to $|\text{CHANGED}| = |\Delta G| + |\Delta\mathbb{C}|$.

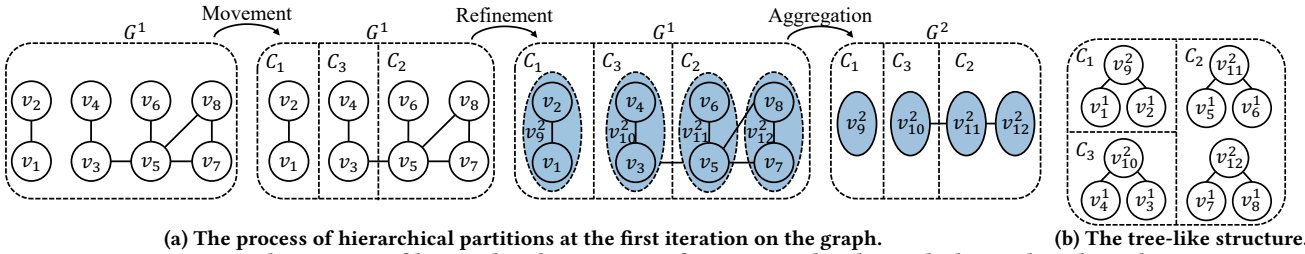
Definition 3 (Boundedness [25, 66]). *An incremental algorithm is bounded if its computational cost can be expressed as a polynomial function of $|\text{CHANGED}|$ and $|\Theta|$. Otherwise, it is unbounded.*

• **Concepts of relative boundedness.** In real-world dynamic graphs, $|\text{CHANGED}|$ is often small, yet some unbounded algorithms can be solved in polynomial time using measures comparable to $|\text{CHANGED}|$, making these algorithms feasible. To assess these incremental algorithms effectively, Fan et al. [25] introduced the concept of relative boundedness, which leverages a more refined cost model called the affected region. Let AFF denote the affected part, the region of the graph actually processed by the incremental algorithm.

Definition 4 (AFF [25]). *Given a graph G, a query Θ, and the input update ΔG to G, AFF signifies the cost difference of the static algorithm between computing $\Theta(G)$ and $\Theta(G \oplus \Delta G)$.*

Unlike CHANGED, AFF captures the concrete portion of the graph touched by an incremental algorithm, providing a tighter bound on its computational cost. This leads to the following definition.

Definition 5 (Relative boundedness [25]). *An incremental graph algorithm is relatively bounded to the static algorithm if its cost is polynomial in $|\Theta|$ and $|\text{AFF}|$.*



(a) The process of hierarchical partitions at the first iteration on the graph. **(b) The tree-like structure.**

Figure 5: The process of hierarchical partitions of Figure 4 at level-1 with the Leiden algorithm.

We now analyze the boundedness of existing incremental Leiden algorithms.

Theorem 1. *When processing an edge deletion or insertion, the increment Leiden algorithms proposed in [70] all cost $O(P \cdot (|V|+|E|))$.*

Table 2: Incremental Leiden algorithms

Method	Time complexity	Relative boundedness
ST-Leiden [70]	$O(P \cdot (\|V\| + \|E\|))$	✗
DS-Leiden [70]	$O(P \cdot (\|V\| + \|E\|))$	✗
DF-Leiden [70]	$O(P \cdot (\|V\| + \|E\|))$	✗
HIT-Leiden	$O(\|N_2(\texttt{CHANGED})\| + \|N_2(\texttt{AFF})\|)$	✓

By Theorem 1, the existing algorithms for maintaining Leiden communities are both unbounded and relatively unbounded as shown in Table 2. They are very costly for large graphs, even with a small update. Following, we review the property of Leiden and then identify AFF of Leiden in the end. f

4.2 Vertex optimality and subpartition γ-density

As shown in the literature [10, 80], if $s^P(\cdot) = f^P(\cdot)$ after P iterations, Leiden is guaranteed to satisfy the following two properties:

- **Vertex optimality:** All the vertices are vertex optimal.
- **Subpartition γ-density:** All the communities are subpartition γ-dense.

To design an efficient and effective maintenance algorithm for Leiden communities, we analyze the behaviors of vertices and communities when the graph changes as follows.

• **Analysis of vertex optimality.** We begin with a key concept.

Definition 6 (Vertex optimality [10]). *A community $C \in \mathbb{C}$ is called vertex optimality if for each vertex $v \in C$ and $C' \in \mathbb{C}$, the modularity gain $\Delta Q(v \to C', \gamma) \leq 0$.*

Next, we introduce an assumption in the maintenance of Louvain communities [69, 97]:

Assumption 1. *The sum of weights of the updated edges is sufficiently small relative to the graph size m.*

Based on Assumption 1, prior studies suggest that when the number of edge updates is small relative to the graph size, three heuristics hold: (1) intra-community edge deletions and inter-community edge insertions could affect vertex-level community membership [69, 97]; (2) Inter-community edge deletions and intra-community edge insertions can be ignored [69, 97]; (3) Vertices directly involved in such edge changes are the most likely to alter their communities [69]. The heuristics are stated in Observation 1, which can be proved based on Definition 6.

Observation 1 ([69]). *Given an intra-community edge deletion $(v_i, v_j, -\alpha)$ or a cross-community edge insertion (v_i, v_j, α), its effect on the community memberships of vertices v_i and v_j can not be ignored.*

We further derive the propagation of community changes from Observation 1.

Lemma 1. *When a vertex v changes its community to C, then the communities of its neighbors not in C in the updated graph could be affected.*

Proof. Assuming v changes its community from C_i to C, there are three cases:

(1) For each neighbor v_i in C_i, the edge (v, v_i) is a *deleted intra-community* edge and an inserted cross-community edge;
(2) For each neighbor v_j in C, the edge (v, v_j) is a deleted cross-community edge and an inserted intra-community edge;
(3) For each other neighbor v_k, edge (v, v_k) is a deleted cross-community edge and an *inserted cross-community* edge.

Since only the first and third cases meet the conditions in Observation 1, all the neighbors of v that are not in C are likely to change their communities. □

Based on these analyses, we develop a novel movement phase, called `inc-movement` in `HIT-Leiden` to preserve vertex optimality, which will be introduced in Section 5.1.

• **Analysis of subpartition γ-density.** For simplified analysis, we first introduce γ-order and γ-connectivity, which are key concepts for defining subpartition γ-density.

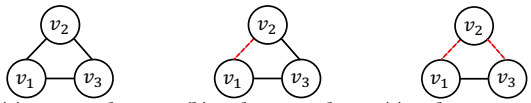
Definition 7 (γ-order). *Given two vertex sequences X and Y of a graph G, let $X \otimes Y$ represent that Y is merged into X such that $2m \cdot w(X, Y) \geq \gamma \cdot d(X) \cdot d(Y)$, where $w(X, Y) = \sum_{v_i \in X} \sum_{v_j \in Y} w(v_i, v_j)$. A γ-order of a vertex sequence $U = \{v_1, \cdots, v_x\}$ represents the merged sequence starting from singleton sequences $\{v_1\}, \cdots, \{v_x\}$.*

We can maintain one γ-order per sub-community from `Leiden`, which is represented by the sequence of vertices merging into the sub-community in `refinement` phase of `Leiden`.

Definition 8 (γ-connectivity [80]). *Given a graph G, a vertex sequence U is γ-connected if U can be generated from at least one γ-order.*

Definition 9 (Subpartition γ-density [80]). *A vertex subsequence $U \subseteq C \in \mathbb{C}$ is subpartition γ-dense if U is γ-connected, and any intermediate vertex sequence X is locally optimized, i.e., $\Delta Q(X \to \emptyset, \gamma) \leq 0$.*

Notably, $\Delta Q(X \to \emptyset, \gamma) \leq 0$ denotes the modularity gain of moving X from C to an empty set, whose calculation follows the same formula as the standard modularity gain in Equation (2).



(a) A triangle. **(b) Delete an edge.** **(c) Delete two edges.**

Figure 6: An example for illustrating subpartition γ-density.

Example 3. *The triangle in Figure 6(a) is subpartition γ-dense with $\gamma = 1$ since there are six different γ-orders. For instance, one is $\{v_3\} \otimes (\{v_1\} \otimes \{v_2\})$, which represents that v_2 is merged into $\{v_1\}$ generating sequence $\{v_1, v_2\}$, and then $\{v_1, v_2\}$ merges into v_3 generating $\{v_1, v_2, v_3\}$. After deleting the edge (v_1, v_2), although $\{v_3\} \otimes (\{v_1\} \otimes \{v_2\})$ is not a γ-order, the update graph is still subpartition γ-dense since $\{v_1\} \otimes (\{v_2\} \otimes \{v_3\})$ is a γ-order in the update graph. After continuing to delete the edge (v_2, v_3), the updated graph is not subpartition γ-dense since v_2 is not connected to v_1 and v_3.*

In essence, each community C (or sub-community S) of Leiden is subpartition γ-dense, since (1) any sub-community in C (or S) is locally optimized, and (2) all sub-communities are γ-connected. Notably, as shown in Figure 3(b), vertex optimality ensures the first condition by design since any sub-community will be a supervertex in `inc-movement` of the next iteration. Thus, we will develop a new refinement algorithm, `inc-refinement`, to preserve γ-connectivity of sub-communities.

Next, we analyze the γ-connectivity property under two kinds of graph updates, i.e., *edge deletion* and *edge insertion*. For any vertex v_i within a sub-community S_i with a γ-order, we denote an intermediate subsequence of the γ-order containing v_i by $I_i \subseteq S_i$, and the subsequence $U_i = I_i \setminus \{v_i\}$ is an intermediate subsequence of the γ-order before merging v_i. For lack of space, all the proofs of lemmas are shown in the appendix of the full version [44] of this paper.

(1) Edge deletion. We consider the deletions of both intra-sub-community edges and cross-sub-community edges:

Lemma 2. *Given an intra-sub-community edge deletion $(v_i, v_j, -\alpha)$, assume v_j is before v_i in the γ-order of the sub-community. The effects of the edge deletion can be described by the following four cases:*

(1) *v_i could be removed from its sub-community only if $\alpha > \frac{2m \cdot w(v_i,U_i) - \gamma \cdot d(v_i) \cdot d(U_i)}{4m + 2w(v_i,U_i)}$;*
(2) *v_j could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2w(v_j,U_j)}$;*
(3) *For any $v_k \in S_i$ $(k \neq i, j)$, it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k,U_k)}$;*
(4) *For any $v_l \notin S_i$, it should be removed from its sub-community if and only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l))}{2w(v_l,U_l)}$.*

Lemma 3. *Given a cross-sub-community edge deletion $(v_i, v_j, -\alpha)$, the effects of the edge deletion can be described by the following four cases:*

(1) *v_i could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{2w(v_i,U_i)}$;*
(2) *v_j holds similar behavior with v_i;*
(3) *For any $v_k \in S_i \cup S_j$ $(k \neq i, j)$, it could be removed its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{2w(v_k,U_k)}$;*
(4) *For any $v_l \notin S_i \cup S_j$, it could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{2w(v_l,U_l)}$.*

(2) Edge insertion. We consider the insertion of edges containing the insertions of both intra-sub-community edges and cross-sub-community edges:

Lemma 4. *Given an edge insertion (v_i, v_j, α), the effects of the edge insertion can be described by the following four cases:*

(1) *v_i could be removed from its sub-community only if $\alpha > \frac{4}{\gamma} m - d(I_i)$ or $\alpha > \frac{2w(v_i,U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$;*
(2) *v_j could be removed from its sub-community, only if $\alpha > \frac{2w(v_j,U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$;*
(3) *For any $v_k \in S_i \cup S_j$ $(k \neq i, j)$, it could be removed from its sub-community, only if $\alpha > \frac{w(v_k,U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2} d(U_k)$;*
(4) *For any $v_l \notin S_i \cup S_j$, it is unaffected.*

Observation 2. *In the refinement phase of Leiden algorithms, each vertex v is likely to be merged into the sub-community (intermediate subsequence U), offering more edge weights $w(v, U)$ and smaller degrees $d(U)$. Therefore, the differences of the values of $d(v)$, $w(v, U)$, and $d(U)$ are very small when the traversal order of vertices to be merged into sub-communities is in ascending order of vertex degree.*

By the above observation, α is unlikely to satisfy the conditions in cases (2)-(4) of Lemma 2, all the cases of Lemma 3, and the conditions in cases (1)-(3) of Lemma 4 when $\alpha \ll m$ (which is often true as stated in Assumption 1). As a result, when designing the maintenance algorithm, we only need to consider the effect of intra-sub-community edge deletions on v_i, which cannot be ignored.

Besides, our experiments show the following observation, which shows that the case (1) of Lemma 2 can also be ignored.

Observation 3. *Given an updated graph with its previous sub-community memberships, for any sub-community S, we treat each connected component in S as a new sub-community. Most of the maintained communities are subpartition γ-dense.*

The above observation holds because Leiden only offers us a γ-order from the refinement phase, and a subgraph often exists with multiple distinct γ-orders as shown in Example 3. Besides, if a vertex is a candidate affecting γ-connectivity, it is often a candidate affecting vertex optimality, e.g., the vertex v_2 in Figure 6(c). In this case, the vertex is likely to change its community before verifying whether the vertex needs to move out of its sub-community. Hence, the case (1) of Lemma 2 can be ignored if the intra-sub-community edge deletion does not cause the sub-community to be disconnected.

Based on Observations 2-3, we develop a novel refinement algorithm, called `inc-refinement`, in `HIT-Leiden`, which will be introduced in Section 5.2. As shown in Figures 13 and Figure 14(b), over 99% maintained communities from `HIT-Leiden` are subpartition γ-dense.

Extension to supergraphs. Changes at the lower level propagate upward to superedge changes in the higher-level supergraph, as Leiden constructs a list of supergraphs in a bottom-up manner. This motivates us to develop an incremental aggregation phase, namely `inc-aggregation`, to compute the superedge changes in Section 5.3.

Example 4. *In Figure 1, communities C_1 and C_2 are treated as supervertices. Deleting an edge $(v_3, v_5, 1)$ and inserting an edge $(v_1, v_3, 1)$ cause v_3 and v_4 to move from C_2 to C_1. This results in the deletion of $(C_2, C_2, -2)$ and insertion of $(C_1, C_1, 2)$ in the supergraph.*

Therefore, we treat each supergraph as a set of facing edge changes from the previous Leiden community and process them using a consistent procedure as shown in Figure 3(b).

Characterization of AFF. Based on these analyses, we define the supervertices that change their communities or sub-communities as the affected area AFF of Leiden.

5 Our HIT-Leiden algorithm

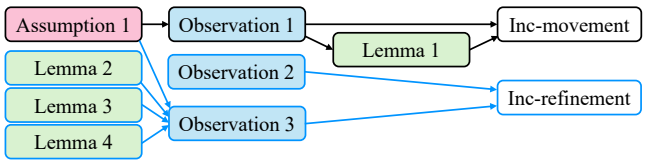


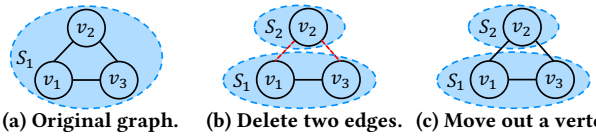
Figure 7: The design rationale for `inc-movement` and `inc-refinement`.

In this section, we first introduce the three key components, namely `inc-movement`, `inc-refinement`, and `inc-aggregation` of our `HIT-Leiden`. Figure 7 shows the assumption, lemmas, and observations used in these components. Then, we present an auxiliary procedure, called deferred update, abbreviated as `def-update`. Afterward, we give an overview of `HIT-Leiden`, and finally analyze the boundedness of `HIT-Leiden`.

5.1 Inc-movement

The goal of `inc-movement` is to preserve vertex optimality. As analyzed in Section 4.2, the endpoints of a deleted intra-community edge or an inserted cross-community edge may affect their community memberships. If an affected vertex changes its community, its neighbors outside the target community may also be affected. Note that any vertex that changes its community has to change its sub-community, since each sub-community is a subset of its community. Hence, sub-community memberships are also considered in `inc-movement`.

We first introduce the data structures used to maintain a dynamic sub-community. According to Observation 3, each connected component of a sub-community is treated as a γ-connected subset. When edge updates or vertex movements split a sub-community into multiple connected components, we re-assign each resulting component as a new sub-community, and the largest sub-community succeeds the original sub-community's ID.



(a) Original graph. (b) Delete two edges. (c) Move out a vertex.

Figure 8: Illustrating the process that a sub-community S_1 is split into two sub-communities S_1 and S_2.

Example 5. *Figure 8 shows the sub-community S_1 is split into two sub-communities $S_1 = \{v_1, v_3\}$ and $S_2 = \{v_2\}$. The component $\{v_1, v_3\}$ retains the original sub-community ID S_1, since it is larger than $\{v_2\}$. The separation can occur either due to the deletion of edges (v_1, v_2) and (v_2, v_3) during graph updates, as shown in Figure 8(b), or due to the removal of vertex v_2 during the movement phase, as shown in Figure 8(c).*

To preserve the structure under such changes, we leverage dynamic connected component maintenance techniques. Various index-based methods have been proposed for this purpose, such as D-Tree [14], DND-Tree [90], and HDT [30]. Let Ψ denote a connected component index, abbreviated as CC-index. The graph G_Ψ stores the subgraph of G consisting only of intra-sub-community edges based on $s(\cdot)$.

Algorithm 2: `Inc-movement`

Input: $G, \Delta G, f(\cdot), s(\cdot), \Psi, \gamma$
Output: Updated $f(\cdot), \Psi, B, K$

1 $A \leftarrow \emptyset, B \leftarrow \emptyset, K \leftarrow \emptyset$;
2 **for** $(v_i, v_j, \alpha) \in \Delta G$ **do**
3 **if** $\alpha > 0$ *and* $f(v_i) \neq f(v_j)$ **then**
4 $A.add(v_i); A.add(v_j)$;
5 **if** $\alpha < 0$ *and* $f(v_i) = f(v_j)$ **then**
6 $A.add(v_i); A.add(v_j)$;
7 **if** $s(v_i) = s(v_j)$ *and* $update_edge\,(G_\Psi, (v_i, v_j, \alpha))$ **then**
8 $K.add(v_i); K.add(v_j)$;
9 **for** $A \neq \emptyset$ **do**
10 $v_i \leftarrow A.pop()$;
11 $C^* \leftarrow argmax_{C \in \mathbb{C} \cup \emptyset} \Delta Q(v_i \rightarrow C, \gamma)$;
12 **if** $\Delta Q(v_i \rightarrow C^*, \gamma) > 0$ **then**
13 $f(v_i) \leftarrow C^*; B.add(v_i)$;
14 **for** $v_j \in N(v_i)$ **do**
15 **if** $f(v_j) \neq C^*$ **then**
16 $A.add(v_j)$;
17 **for** $v_j \in N(v_i) \wedge s(v_i) = s(v_j)$ **do**
18 **if** $update_edge\,(G_\Psi, (v_i, v_j, -w(v_i, v_j)))$ **then**
19 $K.add(v_i); K.add(v_j)$;
20 **return** $f(\cdot), \Psi, B, K$;

Algorithm 2 shows `inc-movement`. Given an updated graph G, a set of graph changes ΔG, community mappings $f(\cdot)$, sub-community mappings $s(\cdot)$, and a CC-index Ψ, it first initializes three empty sets: A, B and K (line 1). Here, A keeps the vertices whose community memberships may be changed, B keeps the vertices that have changed their community memberships, and K records the endpoints on edges whose deletion disconnects the connected component in G_Ψ. Subsequently, vertices involved in intra-community edge deletion or cross-community edge insertion are added to A, and edges in G_Ψ are updated according to intra-sub-community changes (lines 2-7) based on Observations 1 and 3, respectively. If an edge update in G_Ψ causes a connected component to split (i.e., $update_edge(\cdot)$ returns *true*), its endpoints are added to K (line 8). It then processes vertices in A until the set is empty (line 9). For each vertex v_i, it identifies the target community C^* that yields the highest modularity gain (lines 10-11). If $\Delta Q(v_i \rightarrow C^*) > 0$, $f(v_i)$ is updated to C^*, v_i is added into B, and the neighbors of v_i not in C^* are added to A (lines 12-16), which implements the property in Lemma 1. Besides, the intra-sub-community edges involving v_i are deleted from G_Ψ, and the vertices involved in component splits are added to K (lines 17-19). Finally, it returns $f(\cdot)$, Ψ, B, and K (line 20).

5.2 Inc-refinement

As discussed in Section 5.1 and Observation 3, we treat each connected component in G_Ψ maintained in `inc-movement` as a sub-community. Therefore, we design `inc-refinement` for re-assigning

Algorithm 3: `Inc-refinement`

Input: $G, f(\cdot), s(\cdot), \Psi, K, \gamma$
Output: Updated $s(\cdot), \Psi, R$,

1 $R \leftarrow \emptyset$;
2 **for** $v_i \in K$ **do**
3 **if** v_i *is not in the largest connected component of* $s(v)$ **then**
4 Map all vertices in the connected component into a new sub-community and add them into R;
5 **for** $v_i \in R$ **do**
6 **if** v_i *is in singleton sub-community* **then**
7 $\mathcal{T} \leftarrow \{s(v) | v \in N(v_i) \cap f(v_i), \Delta Q(s(v) \rightarrow \emptyset, \gamma) \leq 0\}$;
8 $S^* \leftarrow argmax_{S \in \mathcal{T}} \Delta M(v_i \rightarrow S, \gamma)$;
9 **if** $\Delta M(v_i \rightarrow S^*, \gamma) > 0$ **then**
10 $s(v_i) \leftarrow S^*$;
11 **for** $v_j \in N(v_i)$ **do**
12 **if** $s(v_i) = s(v_j)$ **then**
13 $update_edge\left(G_\Psi, (v_i, v_j, w(v_i, v_j))\right)$;
14 **return** $s(\cdot), \Psi, R$;

each new connected component in G_Ψ as a sub-community. Additionally, we attempt to merge singleton sub-communities whose process is the same as the process of the refinement phase in `Leiden` with G_Ψ maintenance.

Algorithm 3 presents its pseudocode. Given an updated graph G, community mappings $f(\cdot)$ and sub-community mapping $s(\cdot)$, a CC-index Ψ, and a set K, it first initializes R as an empty list to track vertices that have changed their sub-communities (line 1). Note that R is an ordered list sorted in ascending vertex degree mentioned in Observation 2. It then traverses K to identify split connected components in G_Ψ using breadth-first search or depth-first search. If a connected component is not the largest in its original sub-community, all its vertices are re-mapped to a new sub-community, and added to R (lines 2-4). If multiple components tie for the largest component, one of them is randomly selected to represent the original sub-community. For each vertex $v_i \in R$ that is in a singleton sub-community, `inc-refinement` uses a set $\mathcal{T}$ to store the locally optimized neighboring sub-communities of v_i within the same community (lines 5-7). Then, it attempts to reassign v_i to a sub-community $S^* \in \mathcal{T}$, which offers the highest modularity gain to eliminate singleton sub-communities (line 8). Notably, $\Delta M(v_i \rightarrow S, \gamma)$ denotes the modularity gain of moving v_i from $s(v_i)$ to S, whose calculation follows the same formula as the standard modularity gain. If the gain is positive, $s(v_i)$ is updated to S^*, and the corresponding intra-sub-community edges are inserted into G_Ψ (lines 9-13). Finally, `inc-refinement` returns the $s(\cdot)$, Ψ, and R (line 14).

5.3 Inc-aggregation

Given an updated graph G and its edge changes ΔG, modifications to edges and sub-community memberships are reflected as changes to superedges and supervertices in the supergraph H. Let $s_{pre}(\cdot)$ (resp. $s_{cur}(\cdot)$) denotes the vertex-to-supervertex mappings before (resp. after) `inc-refinement`. Any edge change (v_i, v_j, α) in ΔG corresponds to a superedge change $(s_{pre}(v_i), s_{pre}(v_j), \alpha)$ in H, since the weight of a superedge is the sum of weights of edges between their sub-communities. Besides, a vertex v migration from $s_{\text{pre}}(v)$ to $s_{\text{cur}}(v)$ requires updating these weights. Specifically, the original

Algorithm 4: `Inc-aggregation`

Input: $G, \Delta G, s_{pre}(\cdot), s_{cur}(\cdot), R$
Output: $\Delta H, s_{pre}(\cdot)$

1 $\Delta H \leftarrow \emptyset$;
2 **for** $(v_i, v_j, \alpha) \in \Delta G$ **do**
3 $r_i \leftarrow s_{pre}(v_i), r_j \leftarrow s_{pre}(v_j)$;
4 $\Delta H.add((s_i, s_j, \alpha))$;
5 **for** $v_i \in R$ **do**
6 **for** $v_j \in N(v_j)$ **do**
7 **if** $s_{cur}(v_j) = s_{pre}(v_j)$ *or* $i < j$ **then**
8 $\Delta H.add((s_{pre}(v_i), s_{pre}(v_j), -w(v_i, v_j)))$;
9 $\Delta H.add((s_{cur}(v_i), s_{cur}(v_j), w(v_i, v_j)))$;
10 $\Delta H.add((s_{pre}(v_i), s_{pre}(v_i), -w(v_i, v_i)))$;
11 $\Delta H.add((s_{cur}(v_i), s_{cur}(v_i), w(v_i, v_i)))$;
12 **for** $v_i \in R$ **do**
13 $s_{pre}(v_i) \leftarrow s_{cur}(v_i)$;
14 $Compress(\Delta H)$;
15 **return** $\Delta H, s_{pre}(\cdot)$;

sub-community $s_{pre}(v)$ must decrease the superedge weights corresponding to the edge incident to v, and the new sub-community $s_{cur}(v)$ must increase them under the new assignment.

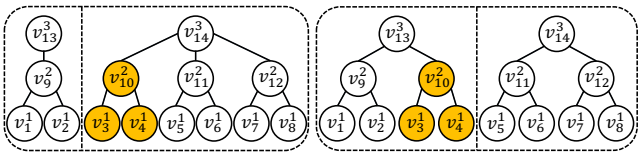
Example 6. *Following Example 4, the initial superedge changes due to edge changes are* $(C_1, C_2, 1)$ *and* $(C_2, C_2, -1)$. *Then, vertices* v_3 *and* v_4 *move from* C_2 *to* C_1, *and there are three cases:*

(1) C_1 *gains edges to the neighbors of* v_3, *resulting in two updates:* $(C_1, C_1, 1)$ *and* $(C_1, C_1, 1)$;
(2) C_2 *loses edges to the neighbor of* v_3 *are* $(C_1, C_2, -1)$ *and* $(C_2, C_2, -1)$;
(3) *The effect of* v_4 *is skipped to avoid duplicate updates, since its only neighbor* v_3 *already changed.*

After compressing the above six superedge changes, we obtain the final superedge changes, which are $(C_1, C_1, 2)$ *and* $(C_2, C_2, -2)$.

Algorithm 4 presents `inc-aggregation`. Initially, the set of changes ΔH of H is empty (line 1). Then, it maps the edge changes ΔG to superedge changes using $s_{pre}(\cdot)$ (lines 2-4). Following, it updates superedges for vertices that switch sub-communities by removing edges from the old community and adding edges to the new one. For any vertex v_i in R, if updates superedges with each neighbor v_j if either $s_{cur}(v_j) = s_{pre}(v_j)$ or $i < j$ to avoid duplicate updates (lines 5-9). Besides, it updates the self-loop for the sub-community of v_i (lines 10-11). Finally, it locally updates $s_{pre}(\cdot)$ to match $s_{cur}(\cdot)$ for the next time step (lines 12-13), and compresses entries by summing the weight of identical superedges in ΔH (line 14).

5.4 Overall HIT-Leiden algorithm



(a) Before maintenance. (b) After maintenance.

Figure 9: The hierarchical partitions changes of Figure 1.

Before presenting our overall HIT-Leiden algorithm, we introduce an optimization technique to further improve the efficiency of

Algorithm 5: `def-update`

Input: $\{f^P(\cdot)\}$, $\{s^P(\cdot)\}$, $\{B^P\}$, P
Output: Updated $\{f^P(\cdot)\}$
1 **for** p *from* P *to* 1 **do**
2 **if** $p \neq P$ **then**
3 **for** $v_i^p \in B^p$ **do**
4 $f^p(v_p^i) = f^{p+1}(s^p(v_p^i))$;
5 **if** $p \neq 1$ **then**
6 **for** $v_i^p \in B^p$ **do**
7 $B^{p-1}.add(s^{-p}(v_i^p))$;
8 **return** $\{f^P(\cdot)\}$;

the vertices' membership update. Specifically, when a supervertex changes its community membership, all the lower-level supervertices associated with it have to update their community membership. As shown in Figure 9, when v_{10}^2 changes its community, v_3^1 and v_4^1 also update their community memberships to the community containing v_{10}^2. However, during the iteration process of `HIT-Leiden`, a supervertex that changes its community does not automatically trigger updates of the community memberships of its constituent lower-level supervertices.

To resolve the above inconsistency, we perform a post-processing step to synchronize the community memberships across all levels, as described in Algorithm 5. Let $\{B^P\}$ denote a sequence of P sets $\{B^1, \cdots, B^P\}$, $\{s^P(\cdot)\}$ denote a sequence of P adajcent-level supervertex mappings $\{s^1(\cdot), \cdots, s^P(\cdot)\}$, and $\{f^P(\cdot)\}$ denote a sequence of P community mappings $\{f^1(\cdot), \cdots, f^P(\cdot)\}$. Note, each B^p in $\{B^P\}$ collects supervertices at level-p whose community memberships have changed, each $s^p(\cdot)$ in $\{s^P(\cdot)\}$ maps from level-p supervertices to their parent supervertices at level-$(p+1)$, and each $f^p(\cdot)$ in $\{f^P(\cdot)\}$ maps from level-p supervertices to their communities. A supervertex is added to B^p for one of two reasons: (1) it changes its community during `inc-movement`, or (2) its higher-level supervertex changes community. Hence, for each level p, `def-update` updates each supervertex in B^p by re-mapping its community membership of its parent using $s^p(\cdot)$ and $f^{p+1}(\cdot)$ when $p \neq P$ (lines 1-4), and adds its constituent vertices to B^{p-1} for the next level updates where $s^{-p}(\cdot)$ is the inverse mapping of $s^p(\cdot)$ when $p \neq 1$ (lines 5-7). This algorithm also supports updating the mappings $\{g^P(\cdot)\}$ from each level supervertex to its level-P ancestor.

• **Overall HIT-Leiden.** After introducing all the key components, we present our overall `HIT-Leiden` in Algorithm 6. The algorithm proceeds over P hierarchical levels, where each level-p operates on a corresponding supergraph G^p. Besides the community membership $f(\cdot)$, `HIT-Leiden` also maintains supergraphs $\{G^P\}$, community mappings $\{f^P(\cdot)\}$, sub-community mappings $\{g^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$ and $\{s_{cur}^P(\cdot)\}$, and CC-indices $\{\Psi^P\}$ to maintain sub-community memberships for each level. Note, $\{s_{pre}^P(\cdot)\}$ are the mappings from the previous time step, and $\{s_{cur}^P(\cdot)\}$ are the in-time mappings to track sub-community memberships as they evolve at the current time step.

Specifically, it initializes $\{s_{cur}^P(\cdot)\} = \{s_{pre}^P(\cdot)\}$. Given the graph change ΔG, it first initializes the first-level update ΔG to ΔG^1 (line 1). It then proceeds through P iterations, each including three phases after updating the supergraph G^p (line 3).

Algorithm 6: `HIT-Leiden`

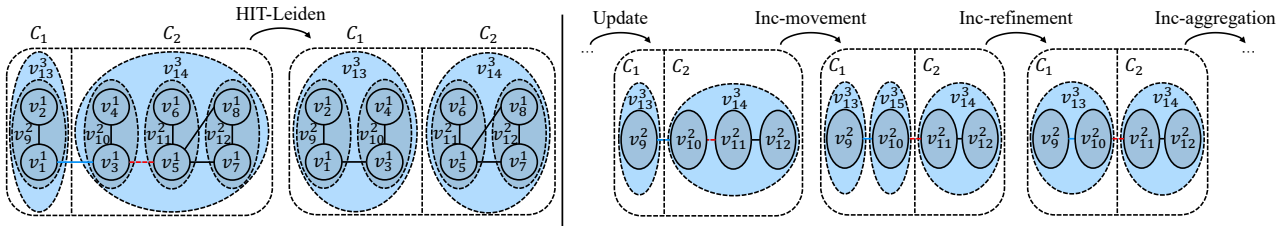
Input: $\{G^P\}$, ΔG, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$,$\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, $\{\Psi^P\}$, P, γ
Output: $f(\cdot)$, $\{G^P\}$, $\{f^P(\cdot)\}$, $\{f^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, $\{\Psi^P\}$
1 $\Delta G^1 \leftarrow \Delta G$;
2 **for** p *from* 1 *to* P **do**
3 $G^p \leftarrow G^p \oplus \Delta G^p$;
4 $f^p(\cdot), \Psi, B^p, K \leftarrow$ `inc-movement`$(G^p, \Delta G^p, f^p(\cdot), s_{cur}^p(\cdot), \Psi, \gamma)$;
5 $s_{cur}^p(\cdot), \Psi, R^p \leftarrow$ `inc-refinement`$(G^p, f^p(\cdot), s_{cur}^p(\cdot), \Psi, K, \gamma)$;
6 **if** $p < P$ **then**
7 $\Delta G^{p+1}, s_{pre}^p(\cdot) \leftarrow$ `inc-aggregation`$(G^p, \Delta G^p, s_{pre}^p(\cdot), s_{cur}^p(\cdot), R^p)$;
8 $\{f^P(\cdot)\} \leftarrow$ `def-update`$(\{f^P(\cdot)\}, \{s_{cur}^P(\cdot)\}, \{B^P\}, P)$;
9 $\{g^P(\cdot)\} \leftarrow$ `def-update`$(\{g^P(\cdot)\}, \{s_{cur}^P(\cdot)\}, \{R^P\}, P)$;
10 $f(\cdot) \leftarrow g^1(\cdot)$;
11 **return** $f(\cdot)$, $\{G^P\}$, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, $\{\Psi^P\}$;

(1) `Inc-movement` (line 4): it re-assigns community memberships of affected vertices to achieve vertex optimality, which yields $f^p(\cdot)$, Ψ, B^p, and K.
(2) `Inc-refinement` (line 5): it re-maps the supervertices of split connected components in Ψ to new sub-communities, producing $s_{cur}^p(\cdot)$, Ψ, and R^p.
(3) `Inc-aggregation` (line 7): it calculates the next level's superedge changes ΔG^{p+1}, and synchronizes $s_{pre}^p(\cdot)$ to match $s_{cur}^p(\cdot)$.

After P iterations, `def-update` (Algorithm 5) synchronizes community mappings $\{f^P(\cdot)\}$ and sub-community mappings $\{g^P(\cdot)\}$ across levels (lines 8-9). The final output $f(\cdot)$ is set to $g^1(\cdot)$ (line 10). Besides, we also return $\{G^P\}$, $\{f^P(\cdot)\}$, $\{g^P(\cdot)\}$, $\{s_{pre}^P(\cdot)\}$, $\{s_{cur}^P(\cdot)\}$, and $\{\Psi^P\}$ for the next graph evolution (line 11).

Example 7. *Consider the result in Figure 4. The graph undergoes an edge deletion* $(v_3^1, v_5^1, -1)$ *and an edge insertions* $(v_1^1, v_3^1, 1)$. *Resulting community and sub-community changes are shown in Figure 10, with hierarchical changes in Figure 9. Take the second iteration as an example. In* `inc-movement`, *the supervertex* v_{10}^2 *is reassigned to* v_{15}^3 *due to disconnection, and migrates from community* C_2 *to* C_1. *In* `inc-refinement`, v_{10}^2 *is merged into* v_{13}^3. *Then,* `inc-aggregation` *calculates superedge changes for level-3, including edge insertion* $(v_{13}^3, v_{13}^3, 2)$ *and edge deletions* $(v_{14}^3, v_{14}^3, -2)$.

• **Complexity analysis.** We now analyze the time complexity of `HIT-Leiden` over P iterations. Let Γ^p denote the set of supervertices involved in superedge changes, and let Λ^p track the supervertices that change their communities or sub-communities at level-p. Therefore, for each level-p, `inc-movement`, `inc-refinement`, and `inc-aggregation` complete in $O(|N_2(\Gamma^p)|+|N_2(\Lambda^p)|)$, $O(|N(\Gamma^p)|+|N(\Lambda^p)|)$, and $O(|N(\Gamma^p)|+|N(\Lambda^p)|)$, respectively. Besides, the time cost of `def-update` is $O\left(\sum_{p=1}^{P}|\Lambda^p|\right)$. Hence, the total time cost of `HIT-Leiden` is $O(\sum_{p=1}^{P}(|N_2(\Gamma^p)|+|N_2(\Lambda^p)|)) = O(|N_2(\texttt{CHANGED})|+|N_2(\texttt{AFF})|)$, as analyzed in Section 4.2. As a result, our `HIT-Leiden` is bounded relative to Leiden.



(a) Community maintain by HIT-Leiden (b) The process of HIT-Leiden in iteration two

Figure 10: An example of HIT-Leiden

6 Experiments

We now present our experimental results. Section 6.1 introduces the experimental setup. Section 6.2 and 6.3 evaluate the effectiveness and efficiency of HIT-Leiden, respectively.

6.1 Setup

Table 3: Datasets used in our experiments.

Dataset	Abbr.	$\|V\|$	$\|E\|$	Timestamp
dblp-coauthor	DC	1.8M	29.4M	Yes
yahoo-song	YS	1.6M	256.8M	Yes
sx-stackoverflow	SS	2.6M	63.4M	Yes
it-2004	IT	41.2M	1.0B	No
risk	RS	201.0M	4.0B	Yes

Datasets. We use four real-world dynamic datasets, including *dblp-coauthor*[1] (academic collaboration), *yahoo-song*[1] (user-song interactions), *sx-stackoverflow*[2] (developer Q&A), and *risk* (financial transactions) provided by Company A. All these dynamic edges are associated with real timestamps. We also use one static dataset *it-2004*[3] (a large-scale web graph), but randomly insert or delete some edges to simulate a dynamic graph. All the graphs are treated as undirected graphs. For each real-world dynamic graph, we collect a sequence of batch updates by sorting the edges in ascending order of their timestamps; for *it-2004*, which lacks timestamps, we randomly shuffle its edge order. Table 3 summarizes the key statistics of the above datasets.

Algorithms. We test the following maintenance algorithms:

- ST-Leiden: A naive baseline that executes the static Leiden algorithm from scratch when the graph changes.
- ND-Leiden: A simple maintenance algorithm in [70], which processes all vertices during the movement phase, initialized with previous community memberships.
- DS-Leiden: A maintenance algorithm based on [70], which uses the delta-screening technique [97] to restrict the number of vertices considered in the movement phase.
- DF-Leiden: An advanced maintenance algorithm from [70], which adopts the dynamic frontier approach [69] to support localized updates.
- HIT-Leiden: Our proposed method.

Dynamic graph settings. As the temporal span varies across datasets (e.g., 62 years for *dblp-coauthor* versus 8 years for *sx-stackoverflow*), we apply a sliding edge window, avoiding reliance on fixed valid time intervals that are hard to standardize. Initially, we construct a static graph using the first 80% of edges. Then, we select a window size $b \in \{10, 10^2, 10^3, 10^4, 10^5\}$, denoting the number of updated edges in an updated batch. Next, we slide this window $r = 9$ times, so we update 9 batches of edges for each dataset. Note that by default, we set $b = 10^3$.

All the algorithms are implemented in C++ and compiled with the gcc 8.3.0 compiler using the -O0 optimization level. We set $\gamma = 1$ and use $P = 10$ iterations. Before running the Leiden community maintenance algorithms, we obtain the communities by running the Leiden algorithm, and HIT-Leiden requires an additional procedure to build auxiliary structures. Due to the limited number of iterations, the community structure has not fully converged, so the maintenance algorithms usually take more time in the first two batches than in other batches. Therefore, we exclude the first two batches from efficiency evaluations. Experiments are conducted on a Linux server running Debian 5.4.56, equipped with an Intel(R) Xeon(R) Platinum 8336C CPU @ 2.30GHz and 2.0 TB of RAM.

6.2 Effectiveness evaluation

To evaluate the effectiveness of different maintenance algorithms, we compare the modularity value and proportion of subpartition γ-dense communities for their returned communities. We also evaluate the long-term effectiveness of community maintenance and present a case study.

• **Modularity.** Figure 11 depicts the average modularity values of all the maintenance algorithms, where the batch size ranges from 10 to 10^5. Figure 12 depicts the modularity value across all the 9 batches, where the batch size is fixed as 1,000. Across all datasets, the expected fluctuation in modularity for ST-Leiden is around 0.02 due to its inherent randomness. These maintenance algorithms achieve equivalent quality in modularity, since the difference in their modularity values is within 0.01. Overall, our HIT-Leiden achieves comparable modularity with other methods.

• **Proportion of subpartition γ-density.** After running HIT-Leiden, for each returned community, we try to re-find its γ-order such that any intermediate vertex set in the γ-order is locally optimized, according to Definition 9. If we can find a valid γ-order for a community, we classify it as a subpartition γ-dense community. We report the proportion of subpartition γ-dense communities in Figure 13. The proportions of subpartition γ-dense communities among these Leiden algorithms are almost 1, and they are within the expected fluctuation (around 0.0001) caused by the inherent randomness of the measure method. Thus, HIT-Leiden achieves a comparable percentage of subpartition γ-density with others.

• **Long-term effectiveness.** To demonstrate the long-term effectiveness of maintaining communities, we enlarge the number r of batches from 9 to 999 and set $b = 10,000$. Figure 14(a)-(c) presents the modularity, proportion of subpartition γ-dense communities, and runtime on the sx-stackoverflow dataset. We observe that incremental Leiden algorithms exhibit higher stability than ST-Leiden

[1] http://konect.cc/networks/
[2] https://snap.stanford.edu/data/
[3] https://networkrepository.com/

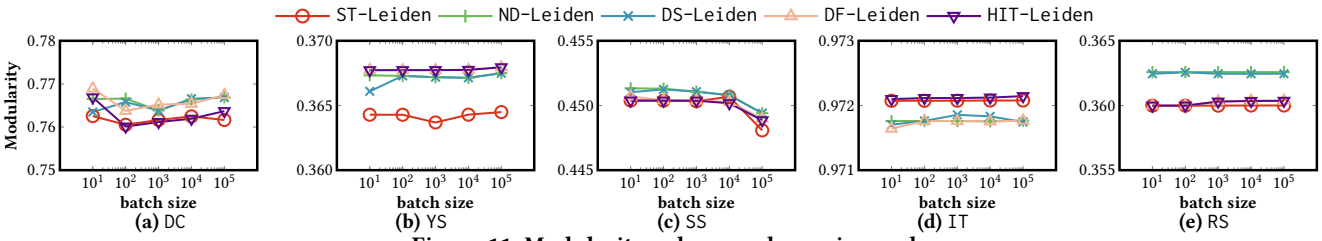


Figure 11: Modularity values on dynamic graphs.

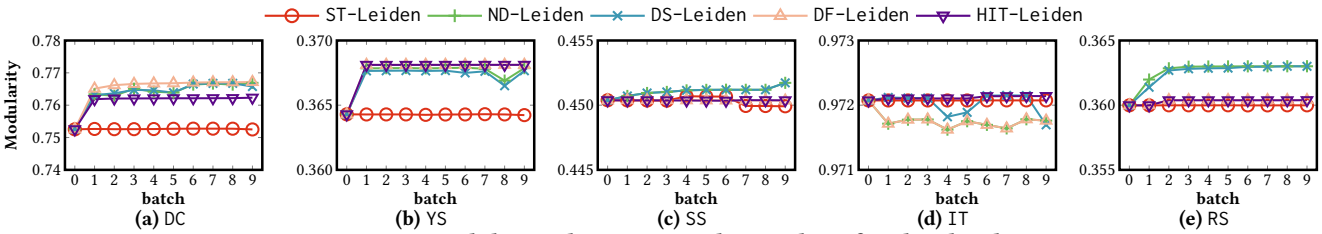


Figure 12: Modularity changes w.r.t. the number of update batches.

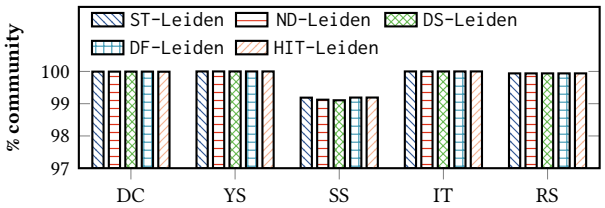


Figure 13: Percentage of subpartition γ-dense communities.

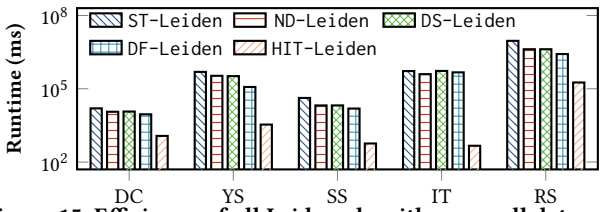


Figure 15: Efficiency of all Leiden algorithms on all datasets.

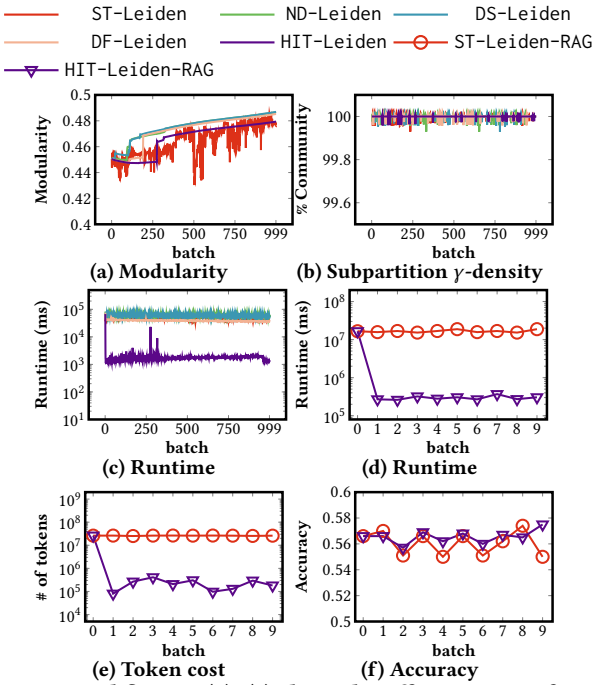
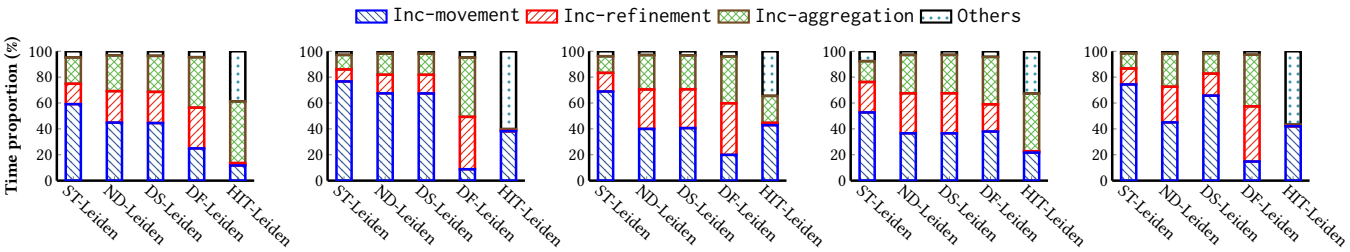


Figure 14: Subfigures (a)–(c) show the effectiveness of HIT-Leiden over 999 update batches, and subfigures (d)–(f) compare ST-Leiden-RAG and HIT-Leiden-RAG over 9 update batches.

in modularity since they use previous community memberships, and HIT-Leiden is faster than other algorithms.

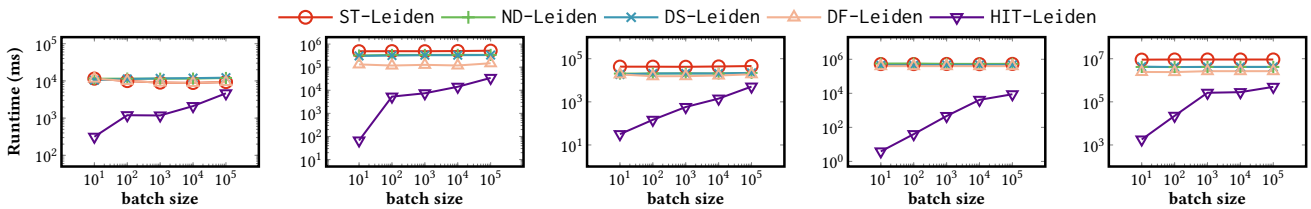
• **A case study.** Our HIT-Leiden has been deployed at Company A to support several real applications. Here, we briefly introduce the application of Graph-RAG. To augment the LLM generation for answering a question, people often retrieve relevant information from an external corpus. To facilitate retrieval, Graph-RAG builds an offline index: It first builds a graph for the corpus, then clusters the graph hierarchically using Leiden, and finally associates a summary for each community, which is generated by an LLM with some token cost. In practice, since the underlying corpus often changes, the communities and their summaries need to be updated as well. Our HIT-Leiden can not only dynamically update the communities efficiently, but also save the token cost since we only need to regenerate the summaries for the updated communities.

To experiment, we use the HotpotQA [95] dataset, which contains Wikipedia-based question-answer (QA) pairs. We randomly select 9,500 articles to build the initial graph, and insert 9 batches of new articles, each with 5 articles. The LLM we use is doubao-1.5-pro-32k. To support a dynamic corpus, we adapt the static Graph-RAG method by updating communities using ST-Leiden and HIT-Leiden, respectively. These two RAG methods are denoted by ST-Leiden-RAG and HIT-Leiden-RAG, respectively. Note that ND-Leiden, DS-Leiden, and DF-Leiden are not fit to maintain the hierarchical communities of Graph-RAG since they lack hierarchical maintenance. We report their runtime, token cost, and accuracy in Figure 14(d)-(f). Clearly, HIT-Leiden-RAG is 56.1× faster than ST-Leiden-RAG. Moreover, it significantly reduces the summary token cost while preserving downstream QA accuracy, since its token cost is only 0.8% of the token cost of ST-Leiden-RAG. Hence, HIT-Leiden is effective for supporting Graph-RAG on a dynamic corpus.



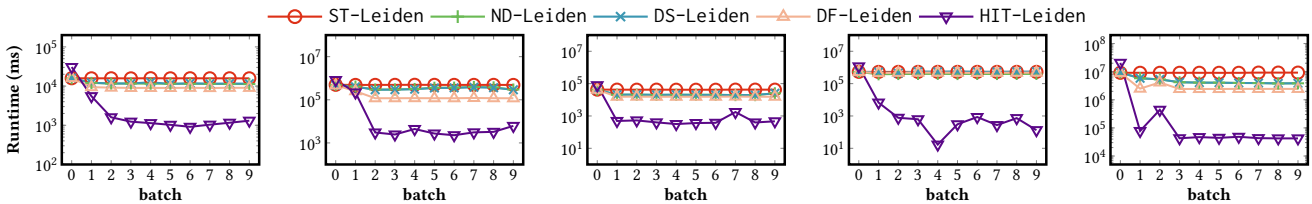
(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 16: Proportion of time cost of each component for the Leiden algorithms on all datasets.



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 17: Runtime on dynamic graphs.



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 18: Runtime w.r.t. the number of update batches.

6.3 Efficiency evaluation

In this section, we first present the overall efficiency results, then analyze the time cost of each component, and finally evaluate the effects of some hyperparameters.

• **Overall results.** Figure 15 presents the overall efficiency results where b is set to its default value 1,000. Clearly, HIT-Leiden achieves the best efficiency on datasets, especially on the it-2004 dataset, since it is up to three orders of magnitude faster than the state-of-the-art algorithms. That is mainly because the ratio of updated edges to total edges in it-2004 is larger than those in dblp-coauthor, yahoo-song, and sx-stackoverflow.

• **Time cost of different components in** HIT-Leiden**.** There are three key components, i.e., inc-movement, inc-refine, and inc-aggregation, in HIT-Leiden. We evaluate the proportion of time cost for each component and present the results in Figure 16. Note that some operations (e.g., def-update in HIT-Leiden) may not be included by the above three components, so we put them into the "Others" component. Notably, in HIT-Leiden, the refinement phase contributes minimally to the overall runtime. Besides, the combined proportion of time spent in its movement and aggregation phase is comparable to that of other algorithms. Inc-movement, inc-refinement, and inc-aggregation consistently outperform their counterparts in other algorithms across all datasets, achieving lower absolute runtime costs according to Figure 15.

• **Effect of** b**.** We vary the batch size $b \in \{10, 10^2, 10^3, 10^4, 10^5\}$ and report the efficiency in Figure 17. We see that HIT-Leiden is up to five orders of magnitude faster than other algorithms. Also, it exhibits a notable increase as b becomes smaller because it is a relatively bounded algorithm. In contrast, ND-Leiden, DS-Leiden, and DF-Leiden still need to process the entire graph when processing a new batch.

• **Effect of** r**.** Recall that after fixing the batch size b, we update the graph for r batches. Figure 18 shows the efficiency, where b is fixed as 1,000, but r ranges from 1 to 9. We observe that the incremental speedup is limited in the first few batches because $P = 10$ is small, and additional iterations may slightly improve the community membership. As a result, all the maintenance algorithms often require more time for the second batch to adjust the community structure. Once high-quality community structure is established, the speedup becomes significant. In addition, HIT-Leiden incurs a slightly higher runtime to record more information and construct the CC-index.

7 Conclusions

In this paper, we develop an efficient algorithm for maintaining Leiden communities in a dynamic graph. We first theoretically analyze the boundedness of existing algorithms and how supervertex behaviors affect community membership under graph update. Building on these analyses, we further develop a relative boundedness algorithm, called HIT-Leiden, which consists of three key components, i.e., inc-movement, inc-refinement, and inc-aggregation. Extensive experiments on five real-world dynamic graphs show that HIT-Leiden not only preserves the properties of Leiden and achieves comparable modularity quality with Leiden, but also runs faster than state-of-the-art competitors. In future work, we will extend our algorithm to handle directed graphs and also evaluate it in a distributed environment.

References

[1] 2020. A single-cell transcriptomic atlas characterizes ageing tissues in the mouse. *Nature* 583, 7817 (2020), 590–595.

[2] Edo M Airoldi, David Blei, Stephen Fienberg, and Eric Xing. 2008. Mixed membership stochastic blockmodels. *Advances in neural information processing systems* 21 (2008).

[3] Arash A Amini, Aiyou Chen, Peter J Bickel, and Elizaveta Levina. 2013. Pseudo-likelihood methods for community detection in large sparse networks. (2013).

[4] Abdelouahab Amira, Abdelouahid Derhab, Elmouatez Billah Karbab, and Omar Nouali. 2023. A survey of malware analysis using community detection algorithms. *Comput. Surveys* 56, 2 (2023), 1–29.

[5] LN Fred Ana and Anil K Jain. 2003. Robust data clustering. In *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2003. Proceedings.*, Vol. 2. IEEE, II–II.

[6] Thomas Aynaud and Jean-Loup Guillaume. 2010. Static community detection algorithms for evolving networks. In *8th international symposium on modeling and optimization in mobile, ad hoc, and wireless networks*. IEEE, 513–519.

[7] Thomas Aynaud and Jean-Loup Guillaume. 2011. Multi-step community detection and hierarchical time segmentation in evolving networks. In *Proceedings of the 5th SNA-KDD workshop*, Vol. 11.

[8] Trygve E Bakken, Nikolas L Jorstad, Qiwen Hu, Blue B Lake, Wei Tian, Brian E Kalmbach, Megan Crow, Rebecca D Hodge, Fenna M Krienen, Staci A Sorensen, et al. 2021. Comparative cellular analysis of motor cortex in human, marmoset and mouse. *Nature* 598, 7879 (2021), 111–119.

[9] Vandana Bhatia and Rinkle Rani. 2018. Dfuzzy: a deep learning-based fuzzy clustering model for large graphs. *Knowledge and Information Systems* 57 (2018), 159–181.

[10] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.

[11] Stefan Boettcher and Allon G Percus. 2002. Optimization with extremal dynamics. *complexity* 8, 2 (2002), 57–62.

[12] Biao Cai, Yanpeng Wang, Lina Zeng, Yanmei Hu, and Hongjun Li. 2020. Edge classification based on convolutional neural networks for community detection in complex network. *Physica A: statistical mechanics and its applications* 556 (2020), 124826.

[13] Tanmoy Chakraborty, Ayushi Dalmia, Animesh Mukherjee, and Niloy Ganguly. 2017. Metrics for community analysis: A survey. *ACM Computing Surveys (CSUR)* 50, 4 (2017), 1–37.

[14] Qing Chen, Sven Helmer, Oded Lachish, and Michael Bohlen. 2022. Dynamic spanning trees for connectivity queries on fully-dynamic undirected graphs. (2022).

[15] Jiafeng Cheng, Qianqian Wang, Zhiqiang Tao, Deyan Xie, and Quanxue Gao. 2021. Multi-view attribute graph convolution networks for clustering. In *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*. 2973–2979.

[16] Yun Chi, Xiaodan Song, Dengyong Zhou, Koji Hino, and Belle L Tseng. 2007. Evolutionary spectral clustering by incorporating temporal smoothness. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 153–162.

[17] Yun Chi, Xiaodan Song, Dengyong Zhou, Koji Hino, and Belle L Tseng. 2009. On evolutionary spectral clustering. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 3, 4 (2009), 1–30.

[18] Wen Haw Chong and Loo Nin Teow. 2013. An incremental batch technique for community detection. In *Proceedings of the 16th international conference on information fusion*. IEEE, 750–757.

[19] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 6 (2004), 066111.

[20] Mário Cordeiro, Rui Portocarrero Sarmento, and Joao Gama. 2016. Dynamic community detection in evolving networks using locality modularity optimization. *Social Network Analysis and Mining* 6 (2016), 1–20.

[21] Ganqu Cui, Jie Zhou, Cheng Yang, and Zhiyuan Liu. 2020. Adaptive graph encoder for attributed graph embedding. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 976–985.

[22] Siemon C de Lange, Marcel A de Reus, and Martijn P van den Heuvel. 2014. The Laplacian spectrum of neural networks. *Frontiers in computational neuroscience* 7 (2014), 189.

[23] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, Vol. 96. 226–231.

[24] Shaohua Fan, Xiao Wang, Chuan Shi, Emiao Lu, Ken Lin, and Bai Wang. 2020. One2multi graph autoencoder for multi-view graph clustering. In *proceedings of the web conference 2020*. 3070–3076.

[25] Wenfei Fan, Chunming Hu, and Chao Tian. 2017. Incremental graph computations: Doable and undoable. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 155–169.

[26] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the web conference 2020*. 2331–2341.

[27] László Gadár and János Abonyi. 2024. Explainable prediction of node labels in multilayer networks: a case study of turnover prediction in organizations. *Scientific Reports* 14, 1 (2024), 9036.

[28] Michael S Haney, Róbert Pálovics, Christy Nicole Munson, Chris Long, Patrik K Johansson, Oscar Yip, Wentao Dong, Eshaan Rawat, Elizabeth West, Johannes CM Schlachetzki, et al. 2024. APOE4/4 is linked to damaging lipid droplets in Alzheimer's disease microglia. *Nature* 628, 8006 (2024), 154–161.

[29] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. 1983. Stochastic blockmodels: First steps. *Social networks* 5, 2 (1983), 109–137.

[30] Jacob Holm, Kristian De Lichtenberg, and Mikkel Thorup. 2001. Poly-logarithmic deterministic fully-dynamic algorithms for connectivity, minimum spanning tree, 2-edge, and biconnectivity. *Journal of the ACM (JACM)* 48, 4 (2001), 723–760.

[31] Ruiqi Hu, Shirui Pan, Guodong Long, Qinghua Lu, Liming Zhu, and Jing Jiang. 2020. Going deep: Graph convolutional ladder-shape networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 2838–2845.

[32] Xiao Huang, Jundong Li, and Xia Hu. 2017. Accelerated attributed network embedding. In *Proceedings of the 2017 SIAM international conference on data mining*. SIAM, 633–641.

[33] Yuting Jia, Qinqin Zhang, Weinan Zhang, and Xinbing Wang. 2019. Communitygan: Community detection with generative adversarial nets. In *The world wide web conference*. 784–794.

[34] Baoyu Jing, Chanyoung Park, and Hanghang Tong. 2021. Hdmi: High-order deep multiplex infomax. In *Proceedings of the web conference 2021*. 2414–2424.

[35] Ravi Kannan, Santosh Vempala, and Adrian Vetta. 2004. On clusterings: Good, bad and spectral. *Journal of the ACM (JACM)* 51, 3 (2004), 497–515.

[36] Brian Karrer and Mark EJ Newman. 2011. Stochastic blockmodels and community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 83, 1 (2011), 016107.

[37] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. 1983. Optimization by simulated annealing. *science* 220, 4598 (1983), 671–680.

[38] Sadamori Kojaku, Giacomo Livan, and Naoki Masuda. 2021. Detecting anomalous citation groups in journal networks. *Scientific Reports* 11, 1 (2021), 14524.

[39] Andrea Lancichinetti and Santo Fortunato. 2009. Community detection algorithms: a comparative analysis. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 80, 5 (2009), 056117.

[40] Ron Levie, Federico Monti, Xavier Bresson, and Michael M Bronstein. 2018. Cayleynets: Graph convolutional neural networks with complex rational spectral filters. *IEEE Transactions on Signal Processing* 67, 1 (2018), 97–109.

[41] Bentian Li, Dechang Pi, Yunxia Lin, and Lin Cui. 2021. DNC: A deep neural network-based clustering-oriented network embedding algorithm. *Journal of Network and Computer Applications* 173 (2021), 102854.

[42] Zhangtao Li and Jing Liu. 2016. A multi-agent genetic algorithm for community detection in complex networks. *Physica A: Statistical Mechanics and its Applications* 449 (2016), 336–347.

[43] Xujian Liang and Zhaoquan Gu. 2025. Fast think-on-graph: Wider, deeper and faster reasoning of large language model on knowledge graph. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24558–24566.

[44] Chunxu Lin, YiXiang Fang, Yumao Xie, Yongming Hu, Yingqian Hu, and Chen Cheng. 2025. Efficient Maintenance of Leiden Communities in Large Dynamic Graphs (full version). https://anonymous.4open.science/r/HIT_Leiden-2DC1.

[45] Yu-Ru Lin, Yun Chi, Shenghuo Zhu, Hari Sundaram, and Belle L Tseng. 2008. Facetnet: a framework for analyzing communities and their evolutions in dynamic networks. In *Proceedings of the 17th international conference on World Wide Web*. 685–694.

[46] Rik GH Lindeboom, Kaylee B Worlock, Lisa M Dratva, Masahiro Yoshida, David Scobie, Helen R Wagstaffe, Laura Richardson, Anna Wilbrey-Clark, Josephine L Barnes, Lorenz Kretschmer, et al. 2024. Human SARS-CoV-2 challenge uncovers local and systemic response dynamics. *Nature* 631, 8019 (2024), 189–198.

[47] Monika Litviňuková, Carlos Talavera-López, Henrike Maatz, Daniel Reichart, Catherine L Worth, Eric L Lindberg, Masatoshi Kanda, Krzysztof Polanski, Matthias Heinig, Michael Lee, et al. 2020. Cells of the adult human heart. *Nature* 588, 7838 (2020), 466–472.

[48] Fanzhen Liu, Zhao Li, Baokun Wang, Jia Wu, Jian Yang, Jiaming Huang, Yiqing Zhang, Weiqiang Wang, Shan Xue, Surya Nepal, et al. 2022. eRiskCom: an e-commerce risky community detection platform. *The VLDB Journal* 31, 5 (2022), 1085–1101.

[49] Fanzhen Liu, Jia Wu, Chuan Zhou, and Jian Yang. 2019. Evolutionary community detection in dynamic social networks. In *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–7.

[50] Yanbei Liu, Xiao Wang, Shu Wu, and Zhitao Xiao. 2020. Independence promoted graph disentangled networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4916–4923.

[51] Linhao Luo, Yixiang Fang, Xin Cao, Xiaofeng Zhang, and Wenjie Zhang. 2021. Detecting communities from heterogeneous graphs: A context path-based graph neural network model. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 1170–1180.

[52] Aaron F McDaid, Derek Greene, and Neil Hurley. 2011. Normalized mutual information to evaluate overlapping community finding algorithms. *arXiv*

preprint arXiv:1110.2515 (2011).
[53] Xiangfeng Meng, Yunhai Tong, Xinhai Liu, Shuai Zhao, Xianglin Yang, and Shaohua Tan. 2016. A novel dynamic community detection algorithm based on modularity optimization. In *2016 7th IEEE international conference on software engineering and service science (ICSESS)*. IEEE, 72–75.
[54] Microsoft. 2025. GraphRAG: A Structured, Hierarchical Approach to Retrieval Augmented Generation. https://microsoft.github.io/graphrag/. Accessed: 2025-03-31.
[55] Ida Momennejad, Hosein Hasanbeig, Felipe Vieira Frujeri, WA Redmond, Hiteshi Sharma, Robert Ness, Nebojsa Jojic, Hamid Palangi, and Jonathan Larson. [n. d.]. Evaluating Cognitive Maps and Planning in Large Language Models with CogEval (Supplementary Materials). ([n. d.]).
[56] Mark EJ Newman. 2004. Fast algorithm for detecting community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 69, 6 (2004), 066133.
[57] Mark EJ Newman. 2006. Finding community structure in networks using the eigenvectors of matrices. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 74, 3 (2006), 036104.
[58] Mark EJ Newman. 2006. Modularity and community structure in networks. *Proceedings of the national academy of sciences* 103, 23 (2006), 8577–8582.
[59] Mark EJ Newman. 2013. Spectral methods for community detection and graph partitioning. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 88, 4 (2013), 042822.
[60] Mark EJ Newman and Michelle Girvan. 2004. Finding and evaluating community structure in networks. *Physical review E* 69, 2 (2004), 026113.
[61] Nam P Nguyen, Thang N Dinh, Sindhura Tokala, and My T Thai. 2011. Overlapping communities in dynamic networks: their detection and mobile applications. In *Proceedings of the 17th annual international conference on Mobile computing and networking*. 85–96.
[62] Nam P Nguyen, Thang N Dinh, Ying Xuan, and My T Thai. 2011. Adaptive algorithms for detecting community structure in dynamic social networks. In *2011 Proceedings IEEE INFOCOM*. IEEE, 2282–2290.
[63] Alexandru Oarga, Matthew Hart, Andres M Bran, Magdalena Lederbauer, and Philippe Schwaller. 2024. Scientific knowledge graph and ontology generation using open large language models. In *AI for Accelerated Materials Design-NeurIPS 2024*.
[64] Shashank Pandit, Duen Horng Chau, Samuel Wang, and Christos Faloutsos. 2007. Netprobe: a fast and scalable system for fraud detection in online auction networks. In *Proceedings of the 16th international conference on World Wide Web*. 201–210.
[65] Songtao Peng, Jiaqi Nie, Xincheng Shu, Zhongyuan Ruan, Lei Wang, Yunxuan Sheng, and Qi Xuan. 2022. A multi-view framework for BGP anomaly detection via graph attention network. *Computer Networks* 214 (2022), 109129.
[66] Ganesan Ramalingam and Thomas Reps. 1996. On the computational complexity of dynamic graph problems. *Theoretical Computer Science* 158, 1-2 (1996), 233–277.
[67] Jörg Reichardt and Stefan Bornholdt. 2006. Statistical mechanics of community detection. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 74, 1 (2006), 016110.
[68] Boyu Ruan, Junhao Gan, Hao Wu, and Anthony Wirth. 2021. Dynamic structural clustering on graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 1491–1503.
[69] Subhajit Sahu. 2024. DF Louvain: Fast Incrementally Expanding Approach for Community Detection on Dynamic Graphs. *arXiv preprint arXiv:2404.19634* (2024).
[70] Subhajit Sahu. 2024. A Starting Point for Dynamic Community Detection with Leiden Algorithm. *arXiv preprint arXiv:2405.11658* (2024).
[71] Subhajit Sahu, Kishore Kothapalli, and Dip Sankar Banerjee. 2024. Fast Leiden Algorithm for Community Detection in Shared Memory Setting. In *Proceedings of the 53rd International Conference on Parallel Processing*. 11–20.
[72] Arindam Sarkar, Nikhil Mehta, and Piyush Rai. 2020. Graph representation learning via ladder gamma variational autoencoders. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 5604–5611.
[73] Akrati Saxena, Yulong Pei, Jan Veldsink, Werner van Ipenburg, George Fletcher, and Mykola Pechenizkiy. 2021. The banking transactions dataset and its comparative analysis with scale-free networks. In *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. 283–296.
[74] Jiaxing Shang, Lianchen Liu, Xin Li, Feng Xie, and Cheng Wu. 2016. Targeted revision: A learning-based approach for incremental community detection in dynamic networks. *Physica A: Statistical Mechanics and its Applications* 443 (2016), 70–85.
[75] Jiaxing Shang, Lianchen Liu, Feng Xie, Zhen Chen, Jiajia Miao, Xuelin Fang, and Cheng Wu. 2014. A real-time detecting algorithm for tracking community structure of dynamic networks. *arXiv preprint arXiv:1407.2683* (2014).
[76] Oleksandr Shchur and Stephan Günnemann. 2019. Overlapping community detection with graph neural networks. *arXiv preprint arXiv:1909.12201* (2019).
[77] Stanislav Sobolevsky, Riccardo Campari, Alexander Belyi, and Carlo Ratti. 2014. General optimization technique for high-quality community detection in complex networks. *Physical Review E* 90, 1 (2014), 012811.
[78] Xing Su, Shan Xue, Fanzhen Liu, Jia Wu, Jian Yang, Chuan Zhou, Wenbin Hu, Cecile Paris, Surya Nepal, Di Jin, et al. 2022. A comprehensive survey on community detection with deep learning. *IEEE transactions on neural networks and learning systems* 35, 4 (2022), 4682–4702.
[79] Tencent. 2019. *Tencent Graph Computing (TGraph) Officially Open Sourced High-Performance Graph Computing Framework: Plato*. Accessed: 2025-04-17.
[80] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports* 9, 1 (2019), 1–12.
[81] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[82] Lewen Wang, Haozhe Zhao, Cunguang Feng, Weiqing Liu, Congrui Huang, Marco Santoni, Manuel Cristofaro, Paola Jafrancesco, and Jiang Bian. 2023. Removing camouflage and revealing collusion: Leveraging gang-crime pattern in fraudster detection. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*. 5104–5115.
[83] Shu Wang, Yixiang Fang, and Wensheng Luo. 2025. Searching and Detecting Structurally Similar Communities in Large Heterogeneous Information Networks. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1425–1438.
[84] Xiao Wang, Nian Liu, Hui Han, and Chuan Shi. 2021. Self-supervised heterogeneous graph neural network with co-contrastive learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 1726–1736.
[85] Wei Xia, Qianqian Wang, Quanxue Gao, Xiangdong Zhang, and Xinbo Gao. 2021. Self-supervised graph convolutional network for multi-view clustering. *IEEE Transactions on Multimedia* 24 (2021), 3182–3192.
[86] Jierui Xie, Mingming Chen, and Boleslaw K Szymanski. 2013. LabelrankT: Incremental community detection in dynamic networks via label propagation. In *Proceedings of the workshop on dynamic networks management and mining*. 25–32.
[87] Jierui Xie and Boleslaw K Szymanski. 2013. Labelrank: A stabilized label propagation algorithm for community detection in networks. In *2013 IEEE 2nd Network Science Workshop (NSW)*. IEEE, 138–143.
[88] Jierui Xie, Boleslaw K Szymanski, and Xiaoming Liu. 2011. Slpa: Uncovering overlapping communities in social networks via a speaker-listener interaction dynamic process. In *2011 ieee 11th international conference on data mining workshops*. IEEE, 344–349.
[89] Yu Xie, Maoguo Gong, Shanfeng Wang, and Bin Yu. 2018. Community discovery in networks with deep sparse filtering. *Pattern Recognition* 81 (2018), 50–59.
[90] Lantian Xu, Dong Wen, Lu Qin, Ronghua Li, Ying Zhang, and Xuemin Lin. 2024. Constant-time Connectivity Querying in Dynamic Graphs. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–23.
[91] Rongbin Xu, Yan Che, Xinmei Wang, Jianxiong Hu, and Ying Xie. 2020. Stacked autoencoder-based community detection method via an ensemble clustering framework. *Information sciences* 526 (2020), 151–165.
[92] Xiaowei Xu, Nurcan Yuruk, Zhidan Feng, and Thomas AJ Schweiger. 2007. Scan: a structural clustering algorithm for networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 824–833.
[93] Liang Yang, Xiaochun Cao, Dongxiao He, Chuan Wang, Xiao Wang, and Weixiong Zhang. 2016. Modularity based community detection with deep learning.. In *IJCAI*, Vol. 16. 2252–2258.
[94] Zhao Yang, René Algesheimer, and Claudio J Tessone. 2016. A comparative analysis of community detection algorithms on artificial networks. *Scientific reports* 6, 1 (2016), 30750.
[95] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600* (2018).
[96] Quanzeng You, Hailin Jin, Zhaowen Wang, Chen Fang, and Jiebo Luo. 2016. Image captioning with semantic attention. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4651–4659.
[97] Neda Zarayeneh and Ananth Kalyanaraman. 2021. Delta-screening: a fast and efficient technique to update communities in dynamic graphs. *IEEE transactions on network science and engineering* 8, 2 (2021), 1614–1629.
[98] Fangyuan Zhang and Sibo Wang. 2022. Effective indexing for dynamic structural graph clustering. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2908–2920.
[99] Meng Zhang, Xingjie Pan, Won Jung, Aaron R Halpern, Stephen W Eichhorn, Zhiyun Lei, Limor Cohen, Kimberly A Smith, Bosiljka Tasic, Zizhen Yao, et al. 2023. Molecularly defined and spatially resolved cell atlas of the whole mouse brain. *Nature* 624, 7991 (2023), 343–354.
[100] Tianqi Zhang, Yun Xiong, Jiawei Zhang, Yao Zhang, Yizhu Jiao, and Yangyong Zhu. 2020. CommDGI: community detection oriented deep graph infomax. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1843–1852.
[101] Xiaotong Zhang, Han Liu, Xiao-Ming Wu, Xianchao Zhang, and Xinyue Liu. 2021. Spectral embedding network for attributed graph clustering. *Neural Networks* 142 (2021), 388–396.

[102] Yao Zhang, Yun Xiong, Yun Ye, Tengfei Liu, Weiqiang Wang, Yangyong Zhu, and Philip S Yu. 2020. SEAL: Learning heuristics for community detection with generative adversarial networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1103–1113.
[103] Han Zhao, Xu Yang, Zhenru Wang, Erkun Yang, and Cheng Deng. 2021. Graph debiased contrastive learning with joint representation clustering.. In *IJCAI*. 3434–3440.
[104] Yingli Zhou, Qingshuo Guo, Yi Yang, Yixiang Fang, Chenhao Ma, and Laks Lakshmanan. 2024. In-depth Analysis of Densest Subgraph Discovery in a Unified Framework. *arXiv preprint arXiv:2406.04738* (2024).
[105] Di Zhuang, J Morris Chang, and Mingchen Li. 2019. DynaMo: Dynamic community detection by incrementally maximizing modularity. *IEEE Transactions on Knowledge and Data Engineering* 33, 5 (2019), 1934–1945.

Appendix

A Proof of lemmas

A.1 Proof of Lemma 2

Proof. We analyze the modularity gain $\Delta M(v \to \emptyset, \gamma)$ for any vertex v, which denotes the modularity gain of moving v from the intermediate subsequence I to $\emptyset$, whose calculation follows the same formula as the standard modularity gain.

According to Definition 8, if $\Delta M(v \to \emptyset, \gamma) > 0$, the intermediate subsequence I could not be γ-connected and v has to leave I. It is different from maintaining vertex optimality (mentioned in Definition 6): If there exists a community C' such that the modularity gain of moving v from its community C to C' is positive, v is not locally optimized and has to be removed from C.

Case 1: v_i is inserted into S after v_j, i.e., $v_j \in I_i$. The old modularity gain $M_{old}(v_i \to \emptyset, \gamma) < 0$ before deletion is:

$$M_{old}(v_i \to \emptyset, \gamma) = -\frac{w(v_i, U_i)}{2m} + \frac{\gamma \cdot d(v_i) \cdot d(U_i)}{4m^2} \leq 0. \tag{3}$$

Where $U_i = I_i \setminus \{v_i\}$. We multiply right side of Equation (3) by $4m^2$ and obtain $X_{(3)}$:

$$X_{(3)} = -2m \cdot w(v_i, U_i) + \gamma \cdot d(v_i) \cdot d(U_i) \leq 0 \tag{4}$$

After the deletion, the new modularity gain $M_{new}(v_i \to \emptyset, \gamma)$ formulates:

$$\begin{aligned}\Delta M_{new}(v_i \to \emptyset, \gamma) = &-\frac{w(v_i, U_i) - 2\alpha}{2(m-\alpha)} \\ &+ \frac{\gamma \cdot (d(v_i) - \alpha) \cdot (d(U_i) - \alpha)}{4(m-\alpha)^2}.\end{aligned} \tag{5}$$

We multiply right side of Equation (5) by $4(m-\alpha)^2$ and obtain $Y_{(5)}$:

$$\begin{aligned}Y_{(5)} &= -2(m-\alpha) \cdot (w(v_i, U_i) - 2\alpha) \\ &\quad + \gamma \cdot (d(v_i) - \alpha) \cdot (d(U_i) - \alpha) \\ &= X_{(3)} + \alpha \cdot (4m + 2w(v_i, U_i) - 4a - \gamma \cdot (d(I_i) - \alpha)) \\ &< X_{(3)} + \alpha \cdot (4m + 2w(v_i, U_i))\end{aligned} \tag{6}$$

If $X_{(3)} + \alpha \cdot (4m + 2w(v_i, U_i)) > 0$, $\Delta M_{new}(v_i \to \emptyset, \gamma)$ could be positive; Otherwise, $\Delta M_{new}(v_i \to \emptyset, \gamma)$ must be non-positive. Therefore, v_i could be removed from its sub-community only if $\alpha > \frac{2m \cdot w(v_i, U_i) - \gamma \cdot d(v_i) \cdot d(U_i)}{4m + 2w(v_i, U_i)}$.

Case 2: v_j is inserted into S before v_i. In this case, we have $v_j \in I_j$, $v_i \notin I_j$, and the edge deletion does not affect intra-edges within U_j. The old modularity gain $M_{old}(v_i \to \emptyset, \gamma) < 0$ before deletion is:

$$M_{old}(v_j \to \emptyset, \gamma) = -\frac{w(v_j, U_j)}{2m} + \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{4m^2}. \tag{7}$$

We multiply right side of Equation (3) by $4m^2$ and obtain $X_{(3)}$:

$$X_{(7)} = -2m \cdot w(v_j, U_j) + \gamma \cdot d(v_j) \cdot d(U_j) < 0 \tag{8}$$

The new modularity gain after the edge deletion becomes:

$$\begin{aligned}\Delta M_{new}(v_j \to \emptyset, \gamma) = &-\frac{w(v_j, U_j)}{2(m-\alpha)} \\ &+ \frac{\gamma \cdot (d(v_j) - \alpha) \cdot d(U_j)}{4(m-\alpha)^2}\end{aligned} \tag{9}$$

We multiply right side of Equation (9) by $4(m-\alpha)^2$ and obtain $Y_{(9)}$:

$$\begin{aligned}Y_{(9)} &= -2(m-\alpha) \cdot w(v_j, U_j) + \gamma \cdot (d(v_j) - \alpha) \cdot d(U_j) \\ &= X_{(7)} + 2\alpha \cdot \big(w(v_j, U_j) - \gamma \cdot d(U_j)\big) \\ &< X_{(7)} + 2\alpha \cdot w(v_j, U_j)\end{aligned} \tag{10}$$

Hence, v_j could be removed from its sub-community only if $\alpha > m - \frac{\gamma \cdot d(v_j) \cdot d(U_j)}{2w(v_j, U_j)}$.

Generalization to other vertices. Consider other vertices v_k and v_l such that $v_k \in S_i$, $k \neq i, j$ and $v_l \notin S_i$. The old modularity gains $M_{old}(v_k \to \emptyset, \gamma) < 0$ and $M_{old}(v_l \to \emptyset, \gamma) < 0$ before deletion are:

$$M_{old}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2m} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4m^2}. \tag{11}$$

$$M_{old}(v_l \to \emptyset, \gamma) = -\frac{w(v_l, U_l)}{2m} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4m^2}. \tag{12}$$

We multiply right side of Equation (11) and (12) by $4m^2$ respectively to obtain $X_{(11)}$ and $X_{(12)}$:

$$X_{(11)} = -2m \cdot w(v_k, U_k) + \gamma \cdot d(v_k) \cdot d(U_k) \leq 0 \tag{13}$$

$$X_{(12)} = -2m \cdot w(v_l, U_l) + \gamma \cdot d(v_l) \cdot d(U_l) \leq 0 \tag{14}$$

After the edge deletion, their new modularity gains are satisfied:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) \leq -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4(m-\alpha)^2}. \tag{15}$$

$$\Delta M_{new}(v_l \to \emptyset, \gamma) = -\frac{w(v_l, U_l)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m-\alpha)^2}. \tag{16}$$

v_k could be merged before v_i and v_j, between v_i and v_j, as well as after v_i and v_j. $\Delta M_{new}(v_k \to \emptyset, \gamma)$ can be formulated as follows:

(1) v_k is merged before v_i and v_j:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot d(U_k)}{4(m-\alpha)^2}; \tag{17}$$

(2)v_k is merged between v_i and v_j:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) - \alpha)}{4(m-\alpha)^2}; \tag{18}$$

(3) v_k is merged after v_i and v_j:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) = -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k) - 2\alpha)}{4(m-\alpha)^2}. \tag{19}$$

Therefore, the equivalent of Equation (15) holds if and only if v_k is merged before v_i and v_j. Then, We multiply right side of Equation (15) and (16) by $4(m-\alpha)^2$ respectively and obtain $Y_{(15)}$ and $Y_{(16)}$:

$$\begin{aligned} Y_{(15)} &= -2(m-\alpha)\cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k)\cdot d(U_k) \\ &= X_{13} + 2\alpha \cdot w(v_k, U_k), \end{aligned} \tag{20}$$

$$Y_{(16)} = X_{14} + 2\alpha \cdot w(v_l, U_l), \tag{21}$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_k)\cdot d(U_k)}{2w(v_k,U_k)}$, v_k could be removed from its sub-community; v_l should be removed from its sub-community if and only if $\alpha > m - \frac{\gamma \cdot d(v_l)\cdot d(U_l)}{2w(v_l,U_l)}$.

□

A.2 Proof of Lemma 3

Proof. We adopt the same notations as in the proof of Lemma 2, with the exception that v_k now denotes a vertex residing in the same sub-community as either v_i or v_j. Based on this setup, the modularity gain after the edge deletion is shown as follows.

Case 1: Consider the endpoint v_i:

$$\begin{aligned} \Delta M_{new}(v_i \to \emptyset, \gamma) &= -\frac{w(v_i, U_i)}{2(m-\alpha)} \\ &\quad + \frac{\gamma \cdot (d(v_i)-\alpha)\cdot d(U_i)}{4(m-\alpha)^2}. \end{aligned} \tag{22}$$

We multiply right side of Equation (22) by $4(m-\alpha)^2$ and obtain $Y_{(22)}$:

$$\begin{aligned} Y_{(22)} &= -2(m-\alpha)\cdot w(v_i, U_i) \\ &\quad + \gamma \cdot (d(v_i)-\alpha)\cdot d(U_i) \\ &= X_{(3)} + \alpha \cdot (2w(v_i, U_i) - \gamma \cdot d(U_i)) \\ &< X_{(3)} + \alpha \cdot 2w(v_i, U_i) \end{aligned} \tag{23}$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_i)\cdot d(U_i)}{2w(v_i,U_i)}$, v_i could be removed from its sub-community. v_j holds similar behavior.

Case 2: Consider the vertex $v_k \in S_i \cup S_j$, $k \neq i, j$:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) \le -\frac{w(v_k, U_k)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_k)\cdot d(U_k)}{4(m-\alpha)^2}. \tag{24}$$

For Equation (24), v_k could be merged before v_i or v_j, as well as after v_i or v_j. Its equivalent holds if and only if v_k is merged before v_i or v_j. We multiply right side of Equation (24) by $4(m-\alpha)^2$ and obtain $Y_{(24)}$:

$$\begin{aligned} Y_{(24)} &= -2(m-\alpha)\cdot w(v_k, U_k) + \gamma \cdot d(v_k)\cdot d(U_k) \\ &= X_{(11)} + 2\alpha \cdot w(v_k, U_k) \end{aligned} \tag{25}$$

Only if $\alpha > m - \frac{\gamma \cdot d(v_k)\cdot d(U_k)}{2w(v_k,U_k)}$, v_k could be removed from its sub-community.

Case 3: Consider the vertex $v_l \notin S_i \cup S_j$:

$$\Delta M_{new}(v_l \to \emptyset, \gamma) = -\frac{w(v_l, U_l)}{2(m-\alpha)} + \frac{\gamma \cdot d(v_l)\cdot d(U_l)}{4(m-\alpha)^2}. \tag{26}$$

Similar to **Case 2**, if and only if $\alpha > m - \frac{\gamma \cdot d(v_l)\cdot d(U_l)}{2w(v_l,U_l)}$, v_l should be removed from its sub-community.

□

A.3 Proof of Lemma 4

Proof. First, we analyze the **insertion of intra-sub-community edges**. We adopt the same notations as in the proof of Lemma 2. Based on this setup, the modularity gain after the edge insertion is shown as follows.

Case 1: Consider the endpoint v_i, which is the latter merged endpoint:

$$\begin{aligned} \Delta M_{new}(v_i \to \emptyset, \gamma) &= -\frac{w(v_i, U_i) + 2\alpha}{2(m+\alpha)} \\ &\quad + \frac{\gamma \cdot (d(v_i)+\alpha)\cdot (d(U_i)+\alpha)}{4(m+\alpha)^2}. \end{aligned} \tag{27}$$

We multiply right side of Equation (27) by $4(m+\alpha)^2$ and obtain $Y_{(27)}$:

$$\begin{aligned} Y_{(27)} &= -2(m+\alpha)(w(v_i, U_i) + 2\alpha) \\ &\quad + \gamma \cdot (d(v_i)+\alpha)\cdot (d(U_i)+\alpha) \\ &= X_{(3)} + \alpha \cdot \big(\gamma \cdot (d(I_i)+\alpha) - 2w(v_i, U_i) - 4\alpha - 4m\big) \\ &< X_{(3)} + \alpha \cdot \big(\gamma \cdot (d(I_i)+\alpha) - 4m\big) \end{aligned} \tag{28}$$

Obviously, only if $\gamma \cdot (d(I_i)+\alpha) - 4m > 0$, i.e., $\alpha > \frac{4}{\gamma}m - d(I_i)$, $Y_{(27)}$ could be positive.

Case 2: Consider the endpoint v_j, which is the former merged endpoint:

$$\begin{aligned} \Delta M_{new}(v_j \to \emptyset, \gamma) &= -\frac{w(v_j, U_i)}{2(m+\alpha)} \\ &\quad + \frac{\gamma \cdot (d(v_j)+\alpha)\cdot d(U_i)}{4(m+\alpha)^2}. \end{aligned} \tag{29}$$

We multiply right side of Equation (29) by $4(m+\alpha)^2$ and obtain $Y_{(29)}$:

$$\begin{aligned} Y_{(29)} &= -2(m+\alpha)\cdot w(v_j, U_j) \\ &\quad + \gamma \cdot (d(v_j)+\alpha)\cdot d(U_j) \\ &= X_{(7)} + \alpha \cdot (\gamma \cdot d(U_j) - w(v_j, U_j)) \\ &< X_{(7)} + \alpha \cdot \gamma \cdot d(U_j) \end{aligned} \tag{30}$$

Only if $\alpha > \frac{2w(v_j,U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$, v_j could be removed from its sub-community.

Case 3: Consider other vertex $v_k \in S_i$, $k \neq i, j$:

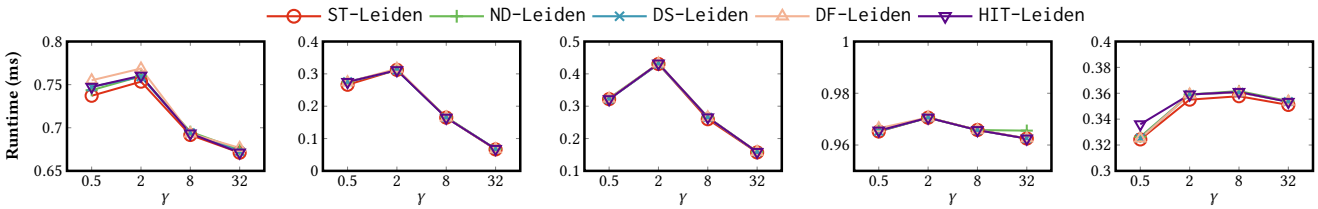
$$\begin{aligned} \Delta M_{new}(v_k \to \emptyset, \gamma) &\le -\frac{w(v_k, U_k)}{2(m+\alpha)} \\ &\quad + \frac{\gamma \cdot d(v_k)\cdot (d(U_k)+2\alpha)}{4(m+\alpha)^2}. \end{aligned} \tag{31}$$

The equivalent of Equation (31) holds if and only if v_k is merged after v_i and v_j. We multiply right side of Equation (31) by $4(m+\alpha)^2$ and obtain $Y_{(31)}$:

$$\begin{aligned} Y_{(31)} &= -2(m+\alpha)\cdot w(v_k, U_k) \\ &\quad + \gamma \cdot d(v_k)\cdot (d(U_k)+2\alpha) \\ &= X_{(11)} + \alpha \cdot \big(2\gamma \cdot d(v_k) - 2w(v_k, U_k)\big) \\ &< X_{(11)} + 2\alpha \cdot \gamma \cdot d(v_k) \end{aligned} \tag{32}$$

Only if $\alpha > \frac{w(v_k,U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2}d(U_k)$, v_k could be removed from its sub-community.

Case 4: Consider other vertex $v_l \notin S_i$:



(a) DC (b) YS (c) SS (d) IT (e) RS

Figure 19: Runtime w.r.t. γ.

$$\Delta M_{new}(v_l \to \emptyset, \gamma) \leq -\frac{w(v_l, U_l)}{2(m+\alpha)} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m+\alpha)^2}. \tag{33}$$

Equation (33) holds if and only if v_j is merged after v_i and v_j. We multiply right side of Equation (33) by $4(m+\alpha)^2$ and obtain $Y_{(31)}$:

$$\begin{aligned} Y_{(33)} &= -2(m+\alpha) \cdot w(v_l, U_l) \\ &\quad + \gamma \cdot d(v_l) \cdot d(U_l) \\ &= X_{(12)} - 2\alpha \cdot w(v_l, U_l) < 0 \end{aligned} \tag{34}$$

v_l is not affected by the intra-sub-community insertion.

Now, we consider the **insertion of cross-sub-community edges**. We adopt the same notations as in the proof of Lemma 3. Based on this setup, the modularity gain after the edge insertion is shown as follows.

Case 5: Consider the endpoint v_i:

$$\Delta M_{new}(v_i \to \emptyset, \gamma) = -\frac{w(v_i, U_i)}{2(m+\alpha)} + \frac{\gamma \cdot (d(v_i)+\alpha) \cdot d(U_i)}{4(m+\alpha)^2}. \tag{35}$$

We multiply right side of Equation (35) by $4(m+\alpha)^2$ and obtain $Y_{(35)}$:

$$\begin{aligned} Y_{(35)} &= -2(m+\alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot (d(v_i)+\alpha) \cdot d(U_i) \\ &= X_{(3)} + \alpha \cdot \big(\gamma \cdot d(U_i) - 2w(v_i, U_i)\big) \\ &< X_{(3)} + \alpha \cdot \gamma \cdot d(U_i) \end{aligned} \tag{36}$$

Only if $\alpha > \frac{2w(v_i,U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$, v_i could be removed from its sub-community. v_j is the same.

Case 6: Consider other vertex $v_k \in S_i \cup S_j$, $k \neq i, j$:

$$\Delta M_{new}(v_k \to \emptyset, \gamma) \leq -\frac{w(v_k, U_k)}{2(m+\alpha)} + \frac{\gamma \cdot d(v_k) \cdot (d(U_k)+\alpha)}{4(m+\alpha)^2}. \tag{37}$$

The equivalent of Equation (37) holds if and only if v_k is merged after v_i or v_j. We multiply right side of Equation (37) by $4(m+\alpha)^2$ and obtain $Y_{(37)}$:

$$\begin{aligned} Y_{(37)} &= -2(m+\alpha) \cdot w(v_i, U_i) \\ &\quad + \gamma \cdot d(v_i) \cdot (d(U_i)+\alpha) \\ &= X_{(3)} + \alpha \cdot \big(\gamma \cdot d(v_i) - 2w(v_i, U_i)\big) \\ &< X_{(3)} + \alpha \cdot \gamma \cdot d(v_i) \\ &< X_{(3)} + 2\alpha \cdot \gamma \cdot d(v_i) \end{aligned} \tag{38}$$

v_k could be removed from its sub-community only if $\alpha > \frac{w(v_k,U_k)}{\gamma d(v_k)} \cdot m - \frac{1}{2}d(U_k)$.

Case 7: Consider other vertex $v_l \notin S_i$:

$$\Delta M_{new}(v_l \to \emptyset, \gamma) \leq -\frac{w(v_l, U_l)}{2(m+\alpha)} + \frac{\gamma \cdot d(v_l) \cdot d(U_l)}{4(m+\alpha)^2}. \tag{39}$$

Equation (39) holds if and only if v_j is merged after v_i and v_j. We multiply right side of Equation (39) by $4(m+\alpha)^2$ and obtain $Y_{(37)}$:

$$\begin{aligned} Y_{(39)} &= -2(m+\alpha) \cdot w(v_l, U_l) \\ &\quad + \gamma \cdot d(v_l) \cdot d(U_l) \\ &= X_{(12)} - 2\alpha \cdot w(v_k, U_l) < 0 \end{aligned} \tag{40}$$

v_l is not affected by the cross-sub-community insertion.

Conclusively, the effects of these edge insertions are:

(1) v_i could be removed from its sub-community only if $\alpha > \frac{4}{\gamma}m - d(I_i)$ or $\alpha > \frac{2w(v_i,U_i)}{\gamma \cdot d(U_i)} \cdot m - d(v_i)$ according to **Case 1 and 5**.
(2) v_j could be removed from its sub-community, only if $\alpha > \frac{2w(v_j,U_j)}{\gamma \cdot d(U_j)} \cdot m - d(v_j)$ according to **Case 2 and 5**.
(3) $v_k \in S_i \cup S_j$ $(k \neq i, j)$ could be removed from its sub-community only if $\alpha > \frac{w(v_k,U_k)}{\gamma \cdot d(v_k)} \cdot m - \frac{1}{2}d(U_k)$ according to **Case 3 and 6**.
(4) $v_l \notin S_i \cup S_j$ is unaffected according to **Case 4 and 7**.

□

B Inaddtional experiments

• **Effect of γ on modularity.** Figure 19 shows the average modularity values for all maintenance algorithms, with the parameter $\gamma \in \{0.5, 2, 8, 32\}$ across all 9 batches, and with the batch size fixed at 1000. Across all datasets, these maintenance algorithms achieve equivalent quality in modularity, since the difference in their modularity values is within 0.01. Overall, our HIT-Leiden still achieves comparable modularity with other methods across different γ.